

Post-quantum transition of device bounded anonymous credentials for the EUDI Wallet

Leonardo Rufini

July 2026

Abstract

In 2026, member states of the European Union are expected to have running their own EUDI wallet applications instances compliant with the ARF architecture guidelines. The goal of the ARF architecture is to provide privacy-preserving anonymous credentials to the users, but the current implementation fails. In the meantime, in 2024 the European Union has published a roadmap for the post quantum digital transition of members digital infrastructures.

No current implementation of anonymous credentials is post quantum secure at the moment, with cryptologist and mathematician in top publications claiming that is not possible to build a post quantum anonymous credential scheme with the actual technologies, given hardware constraints and advanced requirements. So we asked ourselves: is it possible to build a post quantum device bounded anonymous credential scheme with the actual technologies, giving hardware constraints and technical difficulties? The answer is **yes**, it is possible.

Moreover, in this thesis we present a post quantum anonymous credentials model that can be deployed with the actual technologies and legacy infrastructures, with NIST standardized post quantum signature schemes. We went further and implemented the full model and run it into android devices in order to benchmark the performance and the results are surprising. We also present some optimizations that could be implemented in order to further improve the performance of the scheme, and we show how to implement them with the actual technologies.

List of Figures

1	Basic scheme of anonymous credentials.	3
2	Proximity presentation flow (ISO/IEC 18013-5). Adapted from the ARF [15].	5
3	Remote, cross-device presentation flow (OpenID4VP). Adapted from the ARF [15].	6
4	Relying Party authentication, common to the proximity and remote flows. Adapted from the ARF [15].	7
5	Structure and selective-disclosure mechanism of an SD-JWT VC. Adapted from the SD-JWT VC specification [21].	8
6	W3C WebAuthn authentication scheme used for verifiable pseudonyms (passkeys). Adapted from [37].	9
7	EUDI Architecture Reference Framework roles and responsibilities. Adapted from [15].	10
8	Secure element architecture.	19

List of Tables

1	Attestation formats and proof mechanisms supported by the ARF.	7
2	Roles and Responsibilities within the EUDI Wallet Ecosystem	10
3	Prover and Verifier Execution Times Across Various Platforms	13
4	Total processing time comparison of drop-in replacement versus the proposed post-quantum cryptography migration scheme (values in milliseconds).	19
5	Run times and peak memory usage.	20
6	Timing benchmark of an n-block pre-image to the SHA-256 function performed on x86_64 Emerald Rapids.	21

Contents

1	Introduction	3
1.1	Post quantum transition	3
1.2	Overview	4
2	EUropean Digital Identity Wallet	4
2.1	Cryptographic foundations	4
2.2	Functionalities	5
2.2.1	Identification and authentication	5
2.2.2	Exchanging user attributes	6
2.2.3	Qualified electronic signatures (QES)	7
2.2.4	Pseudonyms	8
2.2.5	Payments	8
2.3	Roles	9
2.4	The Concept of Level of Assurance (LoA) High	11
2.4.1	Definition of LoA High	11
2.4.2	Key Security Requirements	11
2.4.3	LoA High in the EUDI Wallet Ecosystem	11
2.5	Zero-knowledge proofs in the ARF	11
2.6	Vulnerabilities	12
3	Literature overview	12
3.1	Anonymous Credentials from ECDSA	12
3.2	Control is Nothing Without Trust a First Look into Digital Identity Wallet Trends	13
3.3	A Brief Note on Cryptographic Pseudonyms for Anonymous Credentials	14
3.4	Device-Bound Anonymous Credentials With(out) Trusted Hardware	14
3.5	Navigating secure storage requirements for EUDI Wallets: a review paper	15
3.6	Privacy Evaluation of the European Digital Identity Wallet’s Architecture and Reference Framework	15
3.7	Protecting Digital Identity Wallet: A Threat Model in the Age of eIDAS 2.0	16
3.8	SoK: Anonymous Credentials for Digital Identity Wallets	17
3.9	Vision: A Modular Framework for Anonymous Credential Systems	17
4	Our solution	18
4.1	Design choices	18
4.2	Implementation	19
4.3	Proof formalization	22
4.3.1	Glossary of Symbols	22
4.3.2	Notation	23
4.3.3	Assumed Components	23
4.3.4	Proof Systems	23
4.3.5	Algorithm 1 — Orchestrator	24
4.3.6	Algorithm 2 — Verifier	25
4.4	Security Analysis	26
4.4.1	MAC witness sharing	26
4.4.2	Compliance with eIDAS 2.0 and LoA High	27
4.4.3	Threat Model	27
5	Results	28
6	Future Work	28
6.1	Formal Treatment and System Completion	28
6.2	Optimization of the Mixed Zero-Knowledge Construction	28
6.3	Extended Credential Functionality	29
6.4	Alternative and Exploratory Research Directions	29
6.5	Evolving the Threat Model	30
7	Conclusion	30

1 Introduction

Anonymous credentials are a powerful tool for preserving privacy in digital interactions between users and service providers. Anonymous credentials were introduced by David Chaum in 1982 [10], and later formalized by Camenisch and Lysyanskaya in 2001 [7, 8]. They allow users to prove their identity or attributes without revealing any additional information about themselves.

The usual model of anonymous credentials consists of three parties:

- The issuer, a central entity who is responsible for issuing credentials to users and eventually revoking them;
- The user, who asks credentials to the issuer and holds them, and eventually use them to prove their identity or attributes to verifiers.
- The verifier, who is responsible for verifying the user’s credentials without learning any additional information about the user.

A basic scheme of how this entity interacts with each other is represented in figure 1.

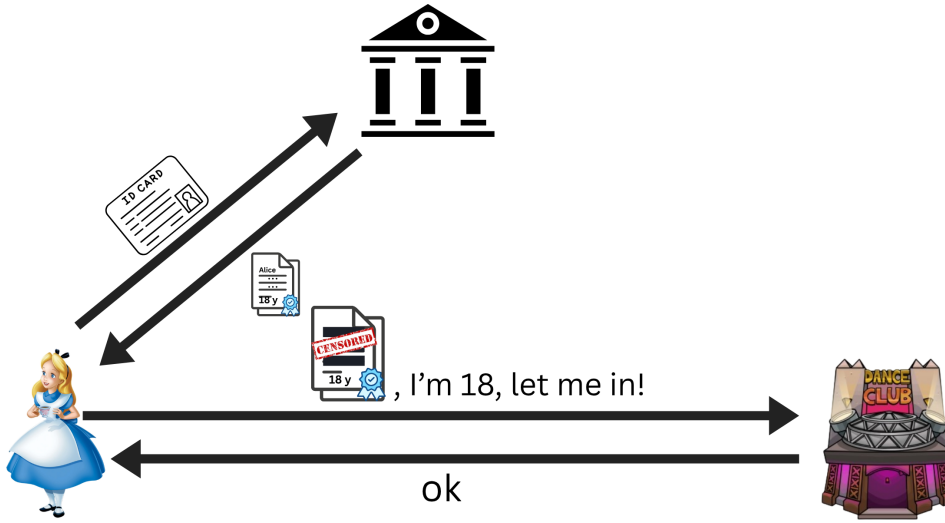


Figure 1: Basic scheme of anonymous credentials.

In the first approach [26], the issuer was signing user’s credentials using a secure protocol that protects user’s attributes. Credential(s) are then showed to a verifier via zero-knowledge proof of knowledge of a credential(s) whose attributes satisfy some predicate.

Systems based on anonymous credentials have been studied and adopted over the years by big tech companies like Microsoft [30], IBM [9], and are in continuous study and development by the scientific community, facing challenges like performance improvements, revocation lists, legacy infrastructures and hardware constraints, and more recently, post quantum security.

1.1 Post quantum transition

The european union on 11 April 2024 has published a roadmap for the post quantum digital transition of members digital infrastructures [16].

The roadmap is a comprehensive plan that outlines the steps that the European Union will take to transition to post-quantum cryptography. The roadmap includes a timeline for the transition, as well as a list of actions that will be taken to ensure a smooth transition.

The timeline is organized into three major phases:

1. **By the end of 2026:** Initial national PQC transition roadmaps have been established by all Member States, PQC transition planning and pilots for high- and medium-risk use cases have been initiated;
2. **By the end of 2030:** The PQC transition for high-risk use cases has been completed, PQC transition planning and pilots for medium-risk use cases have been completed. Quantum-safe software and firmware upgrades are enabled by default.

3. **By the end of 2035:** The PQC transition for medium-risk use cases has been completed. The PQC transition for low-risk use cases has been completed as much as feasible.

As previously said, this roadmap has been published by the European Commission in 2024. The recent advances in the field of post-quantum computing due to hardware improvements, artificial intelligence acceleration and advances in research have made the Q-day closer then expected, and the transition to post-quantum cryptography more urgent.

On 30 March 2026, Google published a drastical advance in post-quantum cryptography: they reduced consistently the amount of time, resources and qbits needed to break elliptic curves based cryptography [5]. In the meanwhile, between April and March 2026, big tech companies like Google [1] and Cloudflare [38] have announced that they will accelerate the transition to post-quantum cryptography, lowering the timeline for the full transition to 2029, instead of 2035.

1.2 Overview

In this thesis, we present a post-quantum anonymous credentials model that can be deployed with the actual technologies and legacy protocols. We require the issuer to adopt ML-DSA NIST standardized post quantum signature scheme, and to slightly leverage the credentials structure in order to accomodate the larger size of post quantum signatures and public keys, as well as modification to the mdoc structure in order to involve a commitment of the device public key instead of the device public key itself.

In the following sections, we are going to present an overview of the EUDI wallet 2, we perform a full ananlysis of the actual state of the art on device bounded anonymous credentials 3, then we explain how our solution faces and solves the challenges, performing an important advance on the actual state of the art 4. We show how we implemented our solution, giving a reference to our github repository 4.2, and we present our results 5, future works that we plan to do ?? and conclusion ??.

2 European Digital Identity Wallet

In 2021, the European Commission proposed to adopt a framework for a European Digital Identity [14]. In 2024, the European Commission decided to establish an Architecture Reference Framework (ARF) [15], which consisted of a set of guidelines and properties that each instance of the wallet application should satisfy. This general framework is needed since the commission left freedom to eacch memeber state to implement their own instance of the application, due to different possible credentials structures (for example, french and german national identity cards have different structures and/or attributes) and infrastructures (for example, some member states have already a national identity card with an embedded chip, while others don't).

In the following section, we are going to analyze the architecture of the EUDI wallet application, to give a complete overview of Architecture Reference Framework, and to analyze the vulnerabilities of the proposed implementation.

2.1 Cryptographic foundations

The entire ecosystem is built on a **Public Key Infrastructure (PKI)**. The main primitives and standards are the following:

- **Elliptic-Curve Cryptography (ECC)** is the preferred choice for mobile-device performance. The relevant presentation standards, ISO/IEC 18013-5 [22] and SD-JWT VC [21], rely respectively on the **COSE** (CBOR Object Signing and Encryption) and **JOSE** (JSON Object Signing and Encryption) frameworks.
- **RSA** is also supported, together with **X.509** certificates [11] and the ETSI standards used for qualified electronic signatures.

For signing documents and transactions the ARF explicitly references the ETSI *AdES* (Advanced Electronic Signature) profiles, namely **XAdES**, **PAdES**, **CAdES** and **JAdES**, depending on the data format being signed.

It is important to stress already at this point that, although the PKI is format-agnostic in principle, the *device-binding* keys – the keys that anchor a credential to a specific device – must reside in the device's secure hardware, which in practice today only supports ECDSA over the NIST P-256 curve. This constraint is central to our solution, discussed in section 4.

2.2 Functionalities

At a high level, the functionalities offered by a Wallet Unit can be grouped into four categories:

- **Secure identification and authentication**, ensuring that users can present person identification data (PID) in a trusted environment, both online and in proximity.
- **Exchanging qualified and non-qualified user attributes** through secure and verifiable electronic attestations of attributes (EAAs).
- **Electronic signing of documents or data**, allowing users to create legally recognised qualified electronic signatures and seals.
- **Generation and use of pseudonyms** for authentication, to enhance privacy and prevent tracking.

A further, emerging use case is the wallet acting as a *strong customer authentication* device for payments, discussed in section 2.2.5.

Now we will discuss in depth each of these functionalities, describing the involved actors and the protocols used to implement them.

2.2.1 Identification and authentication

Description. Users can identify and authenticate to online and offline services while using *selective disclosure* of attributes or attestations.

Actors. User; Wallet Unit (a smartphone application); Relying Party (the service requesting data); PID Provider.

Protocols. Two complementary protocols are used:

- **ISO/IEC 18013-5** [22] for *proximity* flows. The Wallet Unit and the Relying Party set up a communication channel using a QR code or NFC and then communicate over BLE, NFC or Wi-Fi Aware. The standard provides confidentiality and authenticity for all exchanged data, as well as Relying Party authentication. Figure 2 sketches the proximity flow.
- **OpenID4VP** [29] for *remote* (including cross-device) flows. It defines the message structures, transaction flows and an HTTP-based interface for presenting attestations from a Wallet Unit to a Relying Party, again providing confidentiality, authenticity and Relying Party authentication. Figure 3 sketches the remote, cross-device flow.

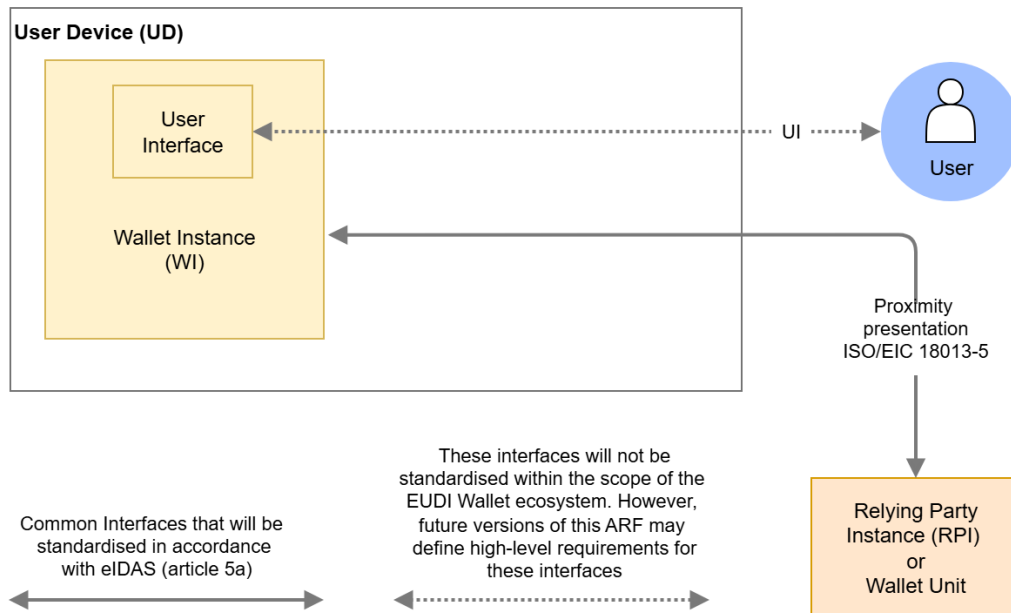


Figure 2: Proximity presentation flow (ISO/IEC 18013-5). Adapted from the ARF [15].

In both protocols, **device binding** is achieved by including a cryptographic public key in the PID or attestation and signing it; the corresponding private key is protected by a Wallet Secure Cryptographic Application/Device (WSCA/WSCD) or keystore inside the Wallet Unit. Although the proximity and remote protocols differ in their details, at a high level they both implement **Relying Party authentication** as illustrated in fig. 4.

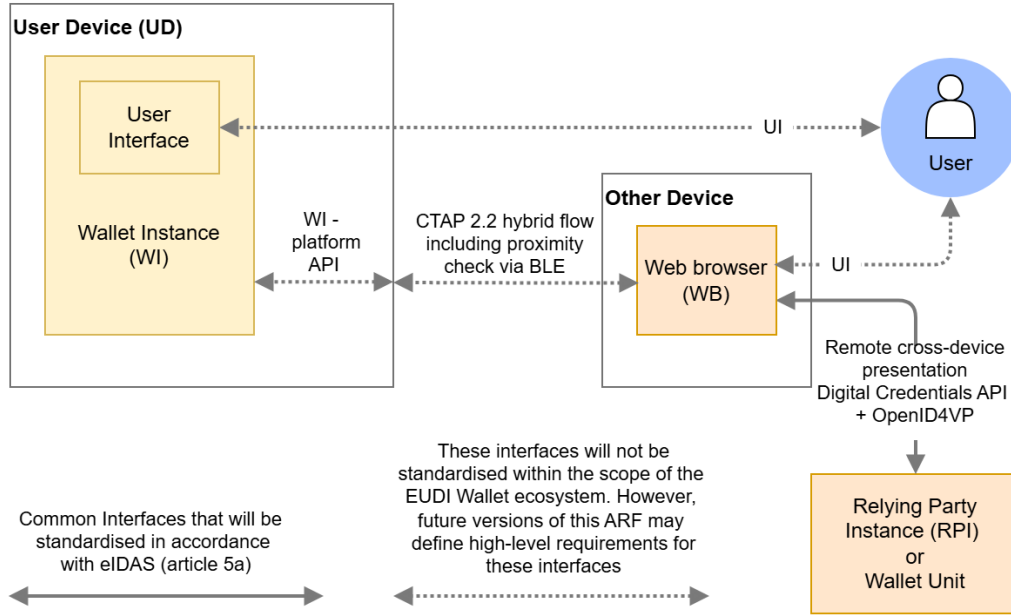


Figure 3: Remote, cross-device presentation flow (OpenID4VP). Adapted from the ARF [15].

2.2.2 Exchanging user attributes

Description. The wallet lets a user request, store and share Electronic Attestations of Attributes – EAA, PuB-EAA and QEAA. The wallet supports *selective disclosure*, so that the user can share only a subset of the attributes contained in an attestation.

Actors. User; Wallet Unit; Relying Party; Attestation Provider (QEAA, PuB-EAA or EAA), the entity issuing the attestation.

Protocol. *SD-JWT VC* [21] (Selectively Disclosable JSON Web Token Verifiable Credentials) is a special form of JWT that is selectively disclosable. It specifies the JSON encoding of attributes and metadata together with rules that prevent collisions of claim names; a proof mechanism that ensures the authenticity and integrity of a PID or attestation while allowing selective disclosure of individual attributes; and an *optional* security mechanism enabling device binding of PIDs and attestations. Figure 5 illustrates the structure of an SD-JWT VC.

How attestations are added. For mobile driving licences (mDL) the Attestation Rulebook explicitly references the ISO/IEC 18013-5 model, the same used for identification. Other attestations are obtained as follows: the user first verifies that the provider is trusted (it appears on an official Trusted List or List of Trusted Entities); the provider authenticates the user, typically reusing the OpenID4VP authentication flow; the provider validates the Wallet Instance through its Wallet Instance Attestation; the user reviews and approves the attestation; and finally the wallet verifies the provider’s signature on the attestation and stores it.

How attestations are stored. The ARF mandates a small set of standardised formats:

- **CBOR**, used by ISO/IEC 18013-5. It is mandatory for mDLs and may be used for any other attestation if the provider remains compliant with the standard.
- **JWT**, used by SD-JWT VC. No credential is required to be expressed in this format, but wallet support is mandatory.
- **W3C Verifiable Credentials Data Model v2.0** [36], an optional format valid only for non-qualified EAAs and still under development.

Table 1 summarises the three formats and their proof mechanisms.

How selective disclosure works. Both ISO/IEC 18013-5 and SD-JWT VC realise selective disclosure through the *salted-hash* technique combined with device and user binding:

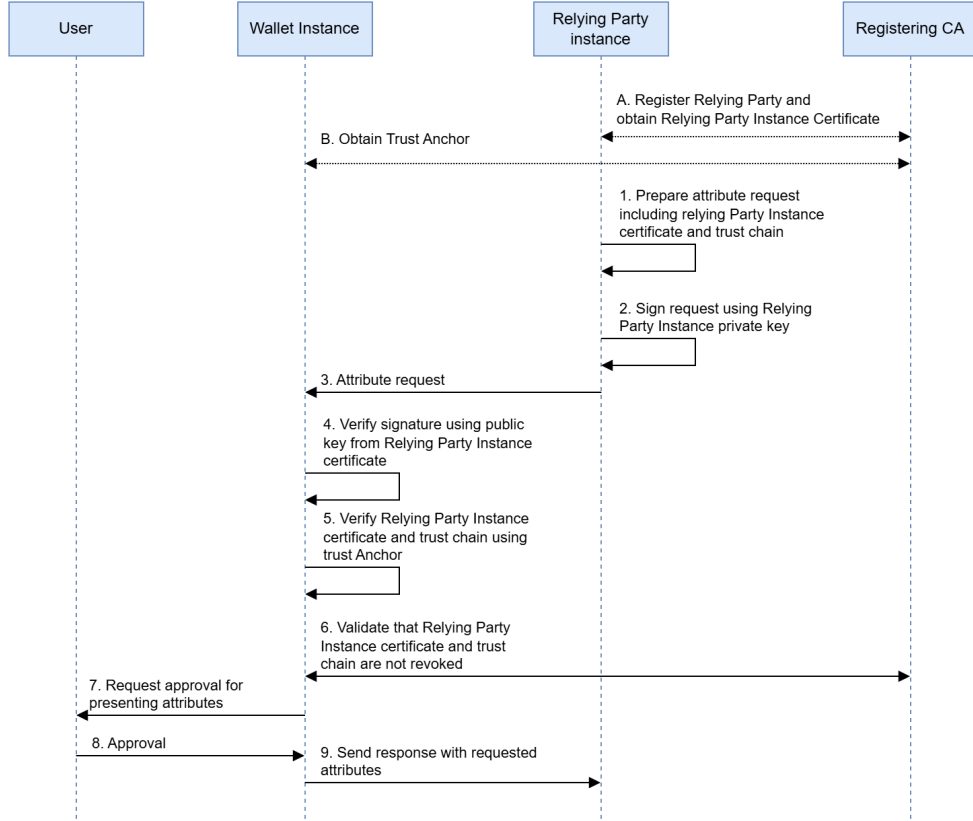


Figure 4: Relying Party authentication, common to the proximity and remote flows. Adapted from the ARF [15].

Table 1: Attestation formats and proof mechanisms supported by the ARF.

Feature	ISO/IEC 18013-5 / 23220-2	SD-JWT VC	W3C VCDM v2.0
Data format	CBOR (binary)	JSON Web Token (JSON)	JSON-LD (linked data)
Proof type	Embedded (salted hashes)	Embedded (salted hashes)	Detached or embedded
Key use case	Proximity (e.g. mDL)	Remote (e.g. remote ident.)	General (requires profiling)
Wallet support	Mandatory	Mandatory	Optional

- **Salted hashes.** The issuer signs a token in which each attribute is replaced by a salted hash of its value rather than its clear text (e.g. the hash of `family_name: Rossi` together with a random salt). To disclose an attribute, the user reveals the clear value and the salt; the verifier recomputes the hash and checks that it matches the one inside the signed token.
- **Device binding.** The attestation embeds a public key whose private counterpart never leaves the WSCD. The provider/verifier issues a random challenge that the device signs, proving that it holds the private key bound to the attestation.
- **User binding.** Before disclosure, the wallet shows the user which attributes are about to be sent and requires local authentication (PIN, biometrics, etc.).

Zero-knowledge proofs, which would allow attributes to be proven without revealing the underlying field, are not yet part of the supported mechanisms; their status is discussed later in the analysis.

2.2.3 Qualified electronic signatures (QES)

Description. The wallet lets users create legally recognised qualified electronic signatures and seals for legally binding transactions and document signing.

Actors. User; Wallet Unit; a *local QSCD* (a local tamper-resistant device that produces signatures) or a remote *QESRC* provider (Qualified Electronic Signature Remote Creation).

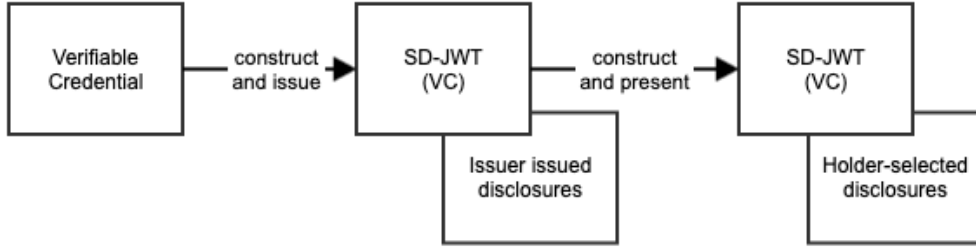


Figure 5: Structure and selective-disclosure mechanism of an SD-JWT VC. Adapted from the SD-JWT VC specification [21].

Protocol. Communication between the Wallet Unit and the QSCD/WSCD takes place through a *Secure Cryptographic Interface*. All parties in the ecosystem that handle cryptographic keys – Wallet Providers, PID and Attestation Providers, Trusted List/LoTE Providers, Providers of Registration Certificates and Access Certificate Authorities – typically rely on a certified **Hardware Security Module (HSM)** to generate and manage private and secret keys. In other words, the security of this functionality rests almost entirely on *hardware* components, which perform key generation and management, user authentication, and the production of signatures and seals; the choice of the actual key-generation protocol is delegated to the wallet implementer.

2.2.4 Pseudonyms

Description. Users can present a pseudonym to a Relying Party in situations where the Relying Party does not need to learn the user’s identity, which helps users avoid being tracked across services.

Actors. User; Wallet Unit; Relying Parties.

Protocol. The ARF distinguishes three kinds of pseudonyms:

- *Verifiable* pseudonyms, which let the user prove ownership;
- *Attested* pseudonyms, a sub-type of verifiable pseudonyms in which a third party attests that the pseudonym belongs to the user;
- *Scope rate-limited* pseudonyms, a sub-type that guarantees the user controls at most a fixed number (the *rate*) of pseudonyms for a given scope.

W3C WebAuthn [37] specifies a class of verifiable pseudonyms known as *passkeys*. The protocol has two phases:

1. **Registration.** The user generates a public/private key pair, stores both keys in a secure device (the *Authenticator*), and registers the public key with the desired Relying Party.
2. **Authentication.** When the user wishes to authenticate, the service sends a challenge consisting of a random value; the user signs it with the private key held by the Authenticator and returns the signature; the service verifies the signature against the registered public key and, if the signature and the origin match, grants access.

Pseudonyms are stored encrypted in a local secure area of the device. Figure 7 illustrates the WebAuthn authentication scheme.

2.2.5 Payments

The EUDI Wallet acts as an *authenticator* that lets the user authorise payments. Unlike, for example, Google Wallet – which typically stores a tokenised version of a physical card – the EUDI Wallet does not store a card number. Instead, it is issued a **Strong User Authentication (SUA)** attestation: a verified digital document that links the Wallet Unit to a specific bank account or payment service. This SUA attestation is *device-bound*, i.e. cryptographically tied to the secure hardware (WSCD/WSCA) of the user’s phone.

When a payment is made, the merchant (acting as a Relying Party) sends the wallet a request containing the transactional data. The user reviews the request on screen and, upon consent, the wallet signs that specific data with its internal keys to authorise the payment. The process is designed to satisfy the **Strong Customer Authentication (SCA)** requirements of the EU payment-services framework

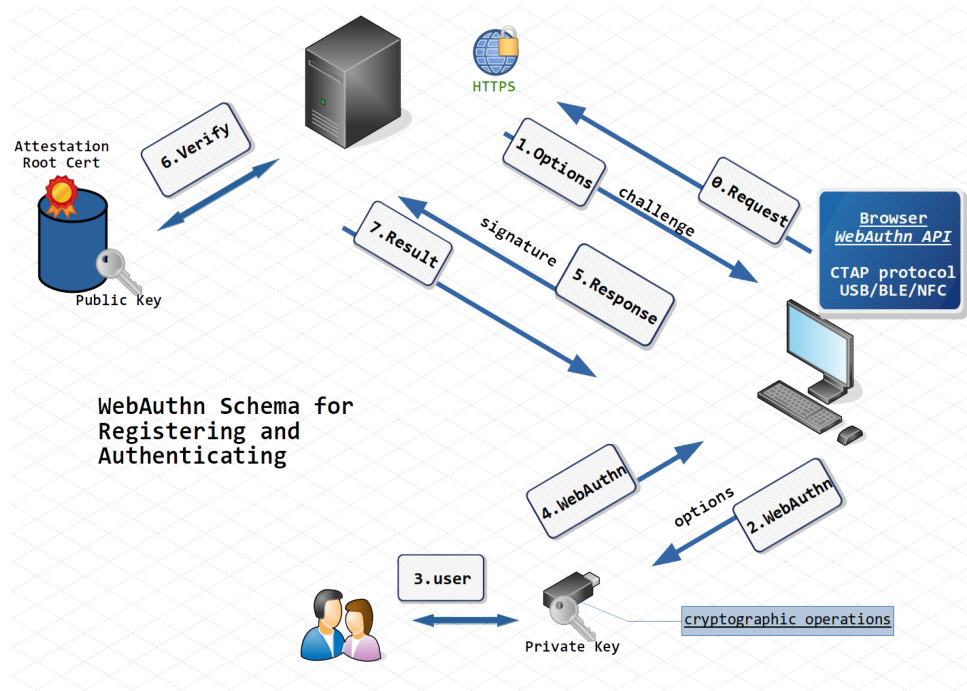


Figure 6: W3C WebAuthn authentication scheme used for verifiable pseudonyms (passkeys). Adapted from [37].

(PSD2/PSD3) [17]. The payment lifecycle therefore has three phases: *registration* (linking the wallet to the bank via an SUA attestation), *authentication* (approving a transaction by signing the amount and payee) and *termination* (unlinking by revoking the SUA attestation).

2.3 Roles

Table 2: Roles and Responsibilities within the EUDI Wallet Ecosystem

Role	Primary Responsibility	Section
User of Wallet Unit	Manage, store, and present PIDs/attestations.	ARF Section 3.2
Wallet Provider	Make the certified Wallet Solution available to Users.	ARF Section 3.3
PID Provider	Issue Person Identification Data (PID) to Users.	ARF Section 3.4
Trusted List/LoTE Provider	Maintain, manage, and publish Trusted Lists and/or Lists of Trusted Entities.	ARF Section 3.5
QEAA Provider	Issue Qualified Electronic Attestations of Attributes (QEAs).	ARF Section 3.6
PuB-EAA Provider	Issue EAAs on behalf of a public sector body.	ARF Section 3.7
EAA Provider	Issue Non-Qualified Electronic Attestations of Attributes (EAAs).	ARF Section 3.8
QESRC Provider	Provide Qualified Electronic Signature Remote Creation services.	ARF Section 3.9
Authentic Source	Act as the definitive repository for specific attributes.	ARF Section 3.10
Relying Party (RP)	Request and receive attributes from a Wallet Unit.	ARF Section 3.11
CAB	Certify Wallet Solutions and audit Trust Service Providers.	ARF Section 3.12
Supervisory Body	Review the proper functioning of ecosystem actors.	ARF Section 3.13
Device Manufacturers	Provide the underlying platform (hardware, OS, secure elements).	ARF Section 3.14
Attestation Scheme Provider	Define and publish the Attestation Rulebooks and schemes.	ARF Section 3.15
NAB	Accredit CABs according to EU regulations.	ARF Section 3.16
Registrar	Manages the registration of Providers and Relying Parties.	ARF Section 3.17
Access CA	Issue access certificates for authentication.	ARF Section 3.18
Provider of Reg. Certs	Issue certificates detailing registration status and scope.	ARF Section 3.19

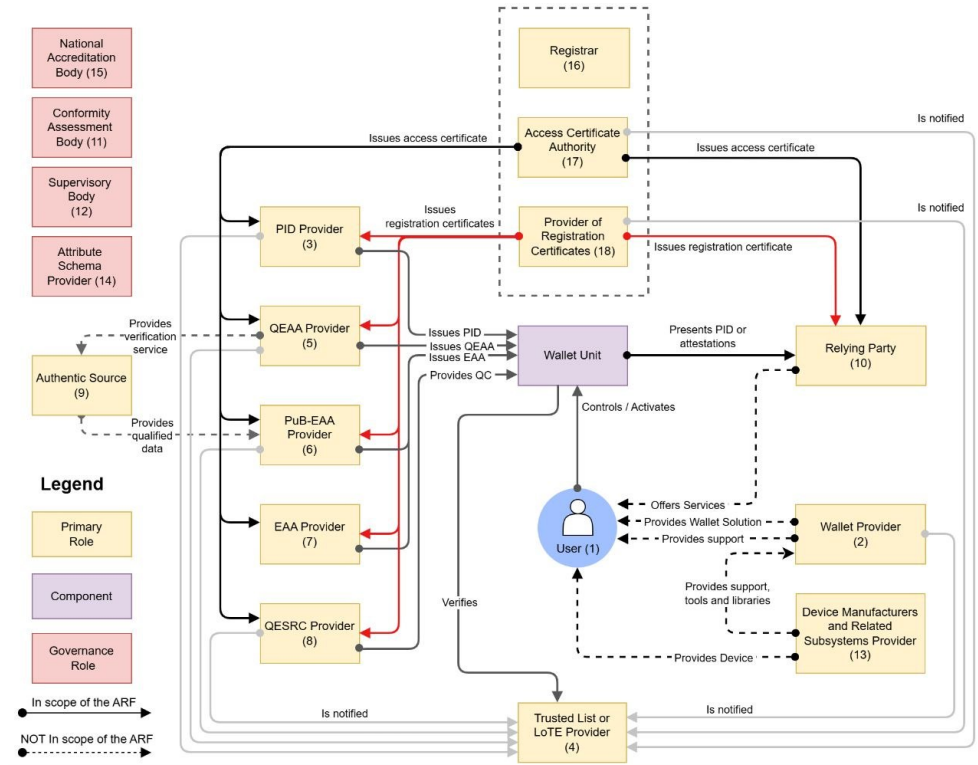


Figure 7: EUDI Architecture Reference Framework roles and responsibilities. Adapted from [15].

2.4 The Concept of Level of Assurance (LoA) High

In the context of the eIDAS 2.0 framework and digital identity systems, the Level of Assurance (LoA) represents the degree of confidence in the claimed identity of a person. It is a critical metric used to determine how much a Relying Party can trust that the user presenting a credential is indeed the person they claim to be.

The technical specifications and procedures for these levels are strictly regulated by Commission Implementing Regulation (EU) 2015/1502.

2.4.1 Definition of LoA High

The LoA High is the highest tier of security defined under the eIDAS regulation. It is designed to protect against attackers with a high attack potential. According to the standard, a "High" level provides a near-absolute degree of certainty in the identity of the user, making it mandatory for sensitive operations such as accessing critical government services or opening financial accounts.

2.4.2 Key Security Requirements

To achieve a High Level of Assurance, several technical and organizational criteria must be met:

- **Secure Cryptographic Hardware:** The cryptographic keys (specifically those associated with the Person Identification Data or PID) must be managed within a Wallet Secure Cryptographic Device (WSCD). This device is legally required to comply with the requirements of LoA High, ensuring that keys are protected against unauthorized duplication or tampering.
- **Common Criteria Certification:** A core determinant for LoA High is the certification of the secure element. It must typically reach a Common Criteria (CC) evaluation level of AVA_VAN.5. This specific level certifies that the device can resist sophisticated, state-level attacks performed by highly capable adversaries.
- **Dynamic Authentication:** The authentication process must be dynamic, meaning the proof generated by the wallet must change for every session. This prevents replay attacks, where an intercepted proof could be used by an unauthorized party.
- **Multi-Factor Authentication (MFA):** LoA High requires the use of at least two authentication factors from different categories, such as "possession" (the device) and "inherence" (biometrics like fingerprints or facial recognition).

2.4.3 LoA High in the EUDI Wallet Ecosystem

Within the EUDI Wallet, the PID is the only attestation that is strictly required by regulation to be issued and managed at LoA High. While other Electronic Attestations of Attributes (EAAs) may use lower levels, the infrastructure of the Wallet must support the High tier to ensure the integrity of the user's core identity. The transition to post-quantum-safe Zero-Knowledge Proofs (ZKPs) is also being explored as a method to maintain this high level of assurance in a future where quantum computers could threaten classical ECC-based security.

2.5 Zero-knowledge proofs in the ARF

Citing ARF developer guide at section [7.4.3.5.3 Zero-knowledge proofs](#), Unlike Relying Party linkability, attestation Provider linkability cannot be fully eliminated when using attestation formats based on salted hashes. The only viable mitigation, they say, is to adopt Zero-Knowledge Proofs (ZKPs) as a verification mechanism instead of relying on salted-attribute hashes.

Furthermore, navigating the ARF developers guide, we found the following statements: Zero-Knowledge Proof (ZKP) mechanisms for verifying personal information are highly promising and essential for ensuring privacy in various use cases. They enable Users to prove statements such as "I am over 18" without disclosing any personal data, offering a robust solution for privacy-preserving authentication and verification.

However, the integration of ZKPs in the EUDI Wallet ecosystem is still under discussion and development, due to the complexity of implementing ZKP solutions in secure hardware and the lack of support in currently available secure hardware (WSCDs). As with Relying Party linkability, organisational and enforcement measures can help deter Attestation Providers from colluding and tracking Users. Additionally, many Attestation Providers are subject to regular audits, making it easier to detect collusion and tracking compared to Relying Parties.

2.6 Vulnerabilities

In this section we are going to analyze the main vulnerabilities of the EUDI wallet found by us, and known in the scientific community.

Linkability Presentations are signed with (and reveal) PKu; PKu and EAArevoc_id persist across sessions; salted hashes stay constant across presentations. Misactors (RPs, APs, WPs, supervisory bodies, external) can correlate repeat interactions and link them to attributable data.

Information leakage through salted-hash selective disclosure Proving a predicate such as "Age over 18" with salted hashes forces disclosure of the WHOLE field, i.e. the full date of birth. Likewise proving nationality may leak the full address / city of residence. The selective-disclosure mechanism lacks granularity: it can only reveal or hide a whole attribute, never a derived predicate. Note on the proposed mitigation: the spec suggests the issuer pre-bakes boolean fields (e.g. "adult: yes/no"). This mitigation is clearly unworkable: it does not scale to arbitrary thresholds (over 19, over 20, ...).

No backup/recovery mechanism The system provides no backup/recovery. If the wallet device is lost, the user loses access to all his credentials, pseudonyms ecc. Moreover, if the device gets broken, Keys are trapped inside the Secure Element and cannot be moved out (QESRC keys cannot leave the SE).

Secure element as a single point of failure All security rests on a single hardware Secure Element. There is no multi-party computation to remove this single point of failure, neither in the ARF nor in the Longfellow (Frigo et al.)

Post-quantum insecurity The entire system is not post-quantum secure actually. Issuer signature, device binding signature, and all the cryptography is based on elliptic curves, which are known to be broken by quantum computers.

Detectability of wallet actions via passive network monitoring EUDIW communication runs over ordinary Internet networks and inherits their threat model. A passive Machine-in-the-Middle can monitor wallet usage without modifying traffic.

A tons of other vulnerabilities have been found and studied in the EUID wallet, some of them will be discussed in the next sections.

3 Literature overview

In this section we are going to analyze the state of the art in the field of anonymous credentials, with a particular focus on device-bound anonymous credentials and on the EUDI wallet application, which are the most relevant for our use case. We will analyze the main papers and solutions proposed in the scientific community, and we will discuss their advantages and disadvantages, as well as their applicability to the EUDI wallet use case.

3.1 Anonymous Credentials from ECDSA

In this [19] paper, the authors M. Frigo and A. Shelat point out the difficulty of building anonymous credentials scheme, given by existing infrastructures that does not allow the implementation of schemes like BBS+, and hardware constraints due to device binding property requirements.

Their solution is to move from the standard model proposed in the ARF architecture with salted hash technique, implementing zero knowledge proofs with Liger framework for arbitrary efficient computation over credentials. The whole system was built upon american credentials, but results were so promising that a team is working on the European implementation of the same framework [13].

The zk technology adopted by Frigo and Shelat to prove the knowledge of a credential is Liger, which is a zk proof system based on the MPC-in-the-head paradigm. The authors managed to optimize the prover time over heavy circuits like ECDSA signature verification circuit or sha256 preimage circuit, via a combination of various techniques, such as sum-check protocols to avoid NTT transforms, reed-solomon error correction codes, cross field MAC and other techniques.

The result is an anonymous credentials systems that can build zero knowledge proof over arbitrary computations in pretty efficient time (look at the table below 3), building a post-quantum secure proof upon post-quantum insecure legacy systems, without requiring any party to change their actual infrastructures. Their non-interactive protocol lacks of non-deniability property over presentations, and their code is quite complex and has been reviewed by experts and bugs have been found, with report available online [35]. In general, being a 20'000 line of codes software it is more prone to bugs and vulnerabilities.

Table 3: Prover and Verifier Execution Times Across Various Platforms

Platform	Prover (ms)	Verifier (ms)
amd_64	526	252
cloud	589	286
Pixel 9	931	508
iPhone 15+	437	229
Mac M4 Air	294	142

3.2 Control is Nothing Without Trust a First Look into Digital Identity Wallet Trends

In this paper [3] by Ansaroudi et al., the authors describe the main functional and non functional requirements for digital identity wallets.

Functional requirements

- The main functional requirements for which they are developed are to store, manage (review, remove), and share (explicitly selecting what data to store/combine and share into/outside the wallet.) the identity or its related data;
- Identity wallets also provide secure storage for cryptographic material associated with the data;
- The wallet may additionally have functionalities for recovery and back up;

The EUDI-ARF covers the aforementioned requirements along with other functionalities such as the creation of (qualified) electronic signatures/seals that demand the provision of interfaces to the services providing the functionality, requiring identification with an eIDAS high LoA.

Non-functional requirements

- **Representation:** Identity subjects must be able to be represented by any number of digital identities;
- **Delegation (of control)/Provability:** Identity subjects must have the ability to control how their identity data is used and exert this control on their choice;
- **Decentralization:** A digital identity data must not be represented, controlled or whatever by a centralized system;
- **Security:** Identity subjects must secure their identity data;
- **Verifiability and authenticity:** Identity subjects are required to provide verifiable proof of the authenticity of their digital identity information;
- **Privacy and minimal disclosure:** Identity subjects must protect the privacy of their digital identity data and share the minimum digital identity data required for any interaction;
- **Transparency:** Identity subjects and all other stakeholders must have easy access to and verify information necessary to understand the incentives, rules, policies, and algorithms under which agents and other components of the ecosystems operate.

The paper then proceeds with a series of comparison between different existing wallets based on the property previously cited.

3.3 A Brief Note on Cryptographic Pseudonyms for Anonymous Credentials

The paper [27] by Mayrhofer, Lehmann and Shelat focuses on how pseudonyms should be realized for the upcoming EUDI wallet, a functionality that the ARF lists (see section 2.2.4) but does not yet pin down at the cryptographic level.

The main argument in favour of pseudonyms is that they make a user *anonymous but still accountable*. Unlike an account created at a relying party with arbitrary, self-claimed attributes, an EUDI pseudonym gives the relying party the guarantee that the pseudonym is backed by a credential that was actually issued to a real end user. A secondary benefit is resistance to bulk account creation by troll or bot farms, since the number of pseudonyms a user can derive for a given service can be bounded. The authors also stress that pseudonyms only add privacy when they are coupled with selective disclosure, otherwise the disclosed attributes would re-identify the user anyway. They classify pseudonyms in three families:

- **Verifiable** pseudonyms, verifiably derived from a user-controlled key;
- **Certified** pseudonyms, verifiably derived from a user holding a certain credential;
- **Scope-exclusive** pseudonyms, derived for a given scope and enforced to be unique per scope.

The actors involved are the holder, the wallet, the holder key, the PID provider and the relying party. Over these actors the authors identify the security and privacy requirements a pseudonym scheme should meet: *soundness* (a holder can create pseudonyms only if it owns an underlying EUDI credential), *unforgeability* (a malicious holder cannot fabricate a pseudonym), *unlinkability*, *unobservability* (the PID provider must not be able to trace the usage of a pseudonym), *selective disclosure*, *scope-exclusiveness* (a relying party is guaranteed that a user can only derive a single pseudonym for its scope) and *transferability* between wallet instances.

Construction. At issuance the credential carries a new high-entropy attribute, the *pseudonym seed* (pns), which is never disclosed directly. To present a pseudonym to a relying party, the wallet computes the pseudonym identifier as $nym = \text{PRF}_{pns}(scp, idx)$, where scp identifies the relying party (e.g. its domain) and idx is an integer that upper-bounds the number of pseudonyms per scope. The holder then sends only the non-identifying attributes and nym , together with a zero-knowledge proof that (i) it holds a valid credential and knows the device holder key when device binding is required, (ii) the credential includes a commitment to pns , and (iii) nym was computed as the PRF keyed on pns with input scp and the optional, possibly hidden, idx . Transferability of a set of pseudonyms then reduces to transferring pns after re-provisioning the holder’s core credentials on the new device.

3.4 Device-Bound Anonymous Credentials With(out) Trusted Hardware

The work of Friedrichs et al. [18] tackles the problem of binding anonymous credentials to a physical device so that they cannot be duplicated or shared among colluding users (the non-transferability property), while preserving the user’s privacy even when the device’s secure element is remote or compromised. As the authors point out, device binding ties a credential to a key held inside a secure element (SE), and every presentation requires a fresh contribution of the SE. Despite being a basic requirement of any real credential, this aspect has received little attention so far.

The paper highlights three challenges that motivate it. First, hardware limitations: many current smartphones do not embed an SE certified at *LoA High*, which pushes the regulation towards remote HSMs that, however, aggravate the privacy risks. Second, trust in the hardware: SEs are black-box components, so a malicious manufacturer or attacker could subvert their code and embed subliminal channels to track users. Third, existing constructions compatible with BBS rely on the honesty of the SE for privacy, which is hard to vet.

To reason about these cases the authors introduce two flavours of the primitive: a *context-aware* variant (DBAC), where the local SE learns the context of the presentation, suitable for trusted local hardware; and a *context-blind* variant (BDBAC), where the SE contributes to the presentation blindly, without learning the context or the disclosed data, which is the model needed for a remote, cloud-based SE. They then define a hierarchy of unlinkability levels, parametrised by how badly the SE is corrupted:

- **Unlink-0:** the SE and the host are fully honest;
- **Unlink-1:** the SE is trusted but not perfectly shielded, i.e. an attacker has temporary access to its interfaces;
- **Unlink-2:** the SE algorithms can be subverted, i.e. the internal code is malicious but stateless;
- **Unlink-3:** the hardware component is actively under the adversary’s real-time control, as required for the cloud HSM model.

On top of this model they give three round-optimal constructions, all using BBS signatures for the credential and a standard signature for the device contribution, proven secure in the AGM+ROM:

- **BBS-Schnorr** uses Schnorr signatures for the binding. It is efficient but reaches only Unlink-1: a malicious SE can exploit the randomness of the Schnorr signature as a subliminal channel to track the user;
- **BBS-BLS** replaces Schnorr with deterministic BLS signatures, reaching Unlink-2 (subversion resistance), because the SE output is unique and cannot hide a channel. The price is that the SE still learns the message it signs;
- **BBS-BlindBLS** uses blind BLS signatures and is the construction meant for a cloud HSM. It reaches Unlink-3 and *SE-obliviousness*, so that not even a malicious remote server learns what the user is signing or who they are.

The merit of this work is that it removes the secure element as a single point of failure that we already met in Longfellow (section 3.1): the binding no longer depends on ECDSA in the SE, so the ad-hoc optimizations that Liger needed are not required, and the compatibility issues of current secure elements are eased. On the negative side, the whole construction is built on BBS and BLS signatures and is therefore *not* post-quantum secure. Moreover, the cloud HSM model is exposed to unavoidable metadata and timing leaks (an observer can monitor when a user is active), it requires connectivity and thus rules out offline usage, and it introduces latency. It is worth noting, however, that some form of connectivity is in any case likely needed at presentation time to check that a credential has not been revoked.

3.5 Navigating secure storage requirements for EUDI Wallets: a review paper

This review by Ebadi Ansaroudi et al. [4] looks at the secure storage problem from a very practical angle: which hardware available on the market today can actually host the keys of an EUDI wallet at the level of assurance the regulation demands. The authors structure the paper around two research questions:

- **RQ1:** do mobile secure storage technologies meet the security requirements of the EUDI wallet, given their advantages in availability and usability?
- **RQ2:** which alternative technologies to mobile solutions can be explored for secure storage in the EUDI wallet?

For RQ1 they survey the local internal WSCD solutions available on mobile devices: Trusted Execution Environments (TEEs), which provide a secure area inside the main processor separated from the operating system (the Android KeyStore and the iOS Secure Enclave), and StrongBox, exposed through the Keystore API and backed by an embedded secure element such as the Titan M2 chip. They then run a market analysis on the Italian market and reach a sobering conclusion: among all vendors, only high-end Samsung and Google Pixel models ship a WSCD that is eIDAS-High compliant and meets the AVA_VAN.5 criteria. Putting the numbers together, roughly 9% of Samsung phones and 1.45% of Pixel phones qualify, for a total of about 10.5% of the devices on the market (potentially around 20.5% if the ongoing evaluation of Apple’s Secure Enclave succeeds). Hardware aside, not every certified WSCD meets the cryptographic requirements either: of the three CC-certified local WSCDs, only KNOX Vault and the Titan M2 chip support ECDSA over the NIST P-256 curve recommended by SOG-IS.

For RQ2 the authors evaluate alternatives, split into internal solutions, embedded secure elements (eSE) and embedded SIMs (eSIM), and external ones, smart cards and backend or remote HSMs, discussing the feasibility and integration potential of each as a consistent secure replacement for the mobile WSCD. The authors are careful to stress that their figures are approximations, limited to the Italian market and based on scarce statistical data.

3.6 Privacy Evaluation of the European Digital Identity Wallet’s Architecture and Reference Framework

Abellán Álvarez et al. [2] carry out a structured privacy evaluation of the EUDI wallet’s ARF (version 1.5) using the LINDDUN threat modeling framework, focusing on three use cases that all involve a high level of assurance: attestation issuance, attestation presentation in remote settings, and attestation presentation in offline settings. The relevant literature is selected through the PRISMA methodology, threats are mapped to concrete harms following Solove’s taxonomy, and the analysis is scoped to the application, data-representation (SD-JWT, mDoc, JSON, CBOR) and session (OpenID4VCI/VP over HTTPS) layers, leaving out the lower transport and network layers.

A central notion in their model is that of *quasi-identifier*: a technical element that, while not directly revealing the user’s legal name, is persistent enough to identify or track them across sessions. The user public key PK_u , the identifiers WUA_{id} and EAA_{revoc_id} , the SD-JWT salted-hash fingerprints that stay constant across presentations, and the persistent certificates used in the ISO/IEC 18013-5 offline flow all fall in this category. On top of these the authors describe seven privacy threats, which we summarize here:

1. **Linkability of quasi-identifiers**: a presentation is signed with, and therefore reveals, the persistent PK_u , and salted hashes do not change across presentations, so repeated interactions can be correlated;
2. **Identifiability between service providers**: a relying party reconstructs a full identity by cooperating with the attestation provider;
3. **Re-identification of pseudonymous identities**: since pseudonym handling is left to each Member State, national legislation could force a relying party or attestation provider to reveal information about the user;
4. **Non-repudiation of wallet actions**: digital signatures bind every action to a single user, removing any plausible deniability;
5. **Excessive data disclosure**: a relying party may request more attributes than it needs, since who decides what is truly necessary is left ambiguous;
6. **Data formats that undermine data protection**: some mandated formats (mDL, ID) carry permanent data that can be stored and later abused;
7. **Detectability via passive network monitoring**: running over ordinary Internet networks, wallet usage can be observed by a passive machine-in-the-middle without modifying any traffic.

Several of these threats are the same ones we already listed among the EUDI wallet vulnerabilities in section 2.6. As mitigations the paper points mainly to privacy-enhancing technologies, and in particular to zero-knowledge proofs, as the way to actually deliver the privacy promises of the EUDI wallet; it also mentions Direct Anonymous Attestation (DAA), which offers selective disclosure and unlinkability but still lacks a specification for digital attestations in this context.

3.7 Protecting Digital Identity Wallet: A Threat Model in the Age of eIDAS 2.0

Sharif et al. [33] provide what they describe as the first detailed threat model of the EUDI wallet. After framing the shift that eIDAS 2.0 pushes, from federated identity management (SAML, OAuth 2.0, OpenID Connect) towards a user-centric, self-sovereign model, they perform their threat modeling combining two frameworks, Microsoft’s STRIDE and LINDDUN, the first for security and the second for privacy. Methodologically this is interesting for us, because the same frameworks can be reused to run a security analysis against a concrete national implementation of the wallet rather than against the generic architecture.

In total the authors identify 49 threats, of which 13 are specific to OpenID4VC (both the issuance and the presentation protocol) and the remaining 36 are specific to the EUDI wallet; only 7 of the latter are detailed in the paper. Among the OpenID4VC attacks they describe abusing the issuer redirect-URI registration to send the user into an attacker-controlled area, intercepting and replaying a request URI to obtain an access token and credentials, brute-forcing request-URI parameters that have insufficient entropy, and reusing stolen or already-spent one-time proofs of wallet control. The seven wallet-specific threats they detail are:

1. **Repudiation** of requested, obtained or revoked credentials, where a malicious holder denies an action by removing the logs;
2. **Credential theft**, where a stolen credential is injected into the communication with the verifier to access a service;
3. **Exploitation of a faulty credential-status check**, where a non-privacy-preserving revocation check lets a curious issuer track the holder’s activity;
4. **Unauthorized VP-token forwarding**, where an attacker forwards a presentation token extracted from a wallet–verifier interaction into their own session to impersonate the holder;
5. **Registration of a non-compliant entity**, abusing a flaw in the certification system to operate a rogue entity;
6. **Exploitation of a faulty data-exchange implementation**, obtaining the holder’s `vp_token` by exploiting a flaw in how the access token is presented;
7. **Data-minimization violation**, requesting and storing more personal data than strictly necessary.

3.8 SoK: Anonymous Credentials for Digital Identity Wallets

This systematization of knowledge by Bormann and Lehmann [6] gives an overview of the two main directions for building an identity system out of anonymous credentials, and is meant to inform the ongoing standardization efforts. The two directions are: dedicated multi-message signature schemes that natively support selective disclosure and efficient zero-knowledge proofs (such as BBS signatures with Schnorr proofs), and general-purpose ZKP systems built on top of legacy ECDSA, which can be deployed over existing infrastructures and standard signatures at the cost of being more complex and less efficient. As the authors recall, both BBS and the zkSNARKs used in the second approach are *post-quantum insecure*.

A useful part of the paper, for our purposes, is the catalogue of features that anonymous credentials can offer. The core ones are:

1. **Selective disclosure:** Ability to reveal only a subset of the credential’s attributes, while keeping the rest hidden;
2. **Multi-show unlinkability:** Relying party to relying party unlinkability ensures that different presentations of credentials by the same user cannot be linked;
3. **Untraceability:** Prevents that a malicious issuer can link the credentials it issued in different presentations;
4. **Single Key Binding:** Many or even all credentials that belong to the same user can be bound to the same user private key to reduce overall complexity;
5. **Presentation freshness:** Presentation of a credential never reveals the original credential, but only a proof thereof. From such a presentation, no further presentation for a different context can be derived;
6. **Certified Pseudonyms:** allow to derive verifiable yet unlinkable pseudonyms from a user’s secret key;

On top of these they discuss more advanced features:

1. **Predicate proofs:** Instead of revealing an exact attribute, proves that a property holds true (ex. age over 18 without revealing birth date);
2. **Composite proofs:** Proof built upon several credentials and more advanced logics;
3. **Deniable presentations:** If the RP forwards the proof to a third party, this does not provide convincing cryptographic evidence of the validity of the user’s attributes;
4. **(Partially-)Blind issuance:** Allows for the issuance of data that stays hidden from the issuer (ex. local university makes central authority signs final degree without revealing student’s name or grades);
5. **Issuer hiding:** Creates a presentation that only reveals membership of a certain trust ecosystem instead of revealing a specific issuer;
6. **Conditional disclosure:** The presentation contains verified but initially protected attributes, which can be discovered by a dedicated party only when needed;

3.9 Vision: A Modular Framework for Anonymous Credential Systems

The vision paper by Lehmann, Sidorenko and Zacharakis [25] starts from a concrete frustration: even though anonymous credentials would be the natural fit for systems like the EUDI wallet or Privacy Pass, neither actually uses them, and both fell back to batch-issuance of one-time tokens, which is expensive and awkward to deploy securely. The authors blame the lack of suitable standards. The schemes that are mature enough, BBS in particular, are being standardized in a fragmented way, with separate efforts in the IETF for the base credential, for pseudonyms, for blind issuance, for Privacy Pass, plus a new ISO draft on attribute-based credentials. The risk they point out is that all these application-tailored standards repeat the same work and end up as slightly different incarnations of the same idea, producing incompatible implementations.

Their answer is to stop standardizing whole systems and standardize *modules* instead. They observe that every anonymous credential system follows the same *sign, commit and prove* blueprint: a core component, that they call CD, is a multi-message signature scheme that supports efficient non-interactive zero-knowledge

proofs of a signature over attributes that can be selectively revealed or kept committed. The commitments are the glue: any feature beyond plain selective disclosure, for example pseudonyms or range proofs, is added as a separate module that operates on those committed attributes. The important design choice is that each module carries its own NIZK proof system and is analyzed in isolation, so its security propagates to the whole system and one can plug optimized proofs into different parts instead of building one monolithic proof for everything. This also gives the framework a crypto-agile flavour, since individual pieces can be swapped without touching the rest.

The part that is most relevant to this thesis is how they handle device binding, a requirement that, as they note, none of the ongoing standardization efforts currently covers. Rather than baking non-transferability into the credential scheme, as the efficient pairing-based proposals do, they keep it as a separate module. The secure element is only ever asked to output a standard signature under a public key that is attested in the credential and revealed only in committed form; the device-binding module then adds a NIZK proving knowledge of a fresh proof-of-possession under that key. The whole point of this decoupling is compatibility with real hardware: today’s secure elements only support ECDSA over P-256, not the pairing-friendly operations those other schemes need, and embedded-hardware signature schemes are notoriously hard, if not impossible, to update, whereas the signature on the main credential is not subject to the same constraint, exactly the tension we raised in section 2.1. As a concrete instantiation they describe *BBS-ECDSA*: BBS for the multi-message signature, with lightweight Schnorr proofs and Pedersen commitments to bridge to the other modules, and the Schnorr-based ZKAttest/CDLS protocol for the proof-of-possession, extended with an extra proof to bridge the two different elliptic curves used by BBS and ECDSA. This BBS-ECDSA combination was actually proposed and implemented by Ubique for the German EUDI innovation challenge and later improved by Dock Labs. It is worth stressing that this construction, like the rest of the literature in this chapter, remains classical: it solves the legacy-hardware compatibility problem but not the post-quantum one.

4 Our solution

In this section, we are going to describe our solution to the problem of building anonymous credentials for the EUDI wallet application, focusing on Post-Quantum Security and on the Device Binding property.

4.1 Design choices

The first challenge we had to face while designing our solution was whether to be compliant with the actual credential’s issuer infrastructures, as well as with the actual standards for credential presentation, credential structures and so on, and to be compliant with the actual hardware constraints of the device that will be used as wallet, or to implement a brand new system that would be more secure and efficient, but would require a complete redesign of the actual infrastructures and standards, as well as new logic and security analysis.

We decided to be compliant with the actual infrastructures and standards, in particular keeping the device binding signature as an ECDSA signature over the session transcript, and just requiring the issuer to sign the credentials with a zk-friendly version of dilithium, a post quantum signature scheme. This is necessary to be post quantum secure, otherwise a quantum adversary could just read the issuer’s public key, obtain the private key via Shor’s algorithm and forge credentials at will.

Moreover in our implementation we will show that protecting device public key with respect to a quantum adversarial issuer and protecting issuer’s key with respect to a quantum adversarial user, while keeping previous presentation scheme requires only to change proof system (which has been realized in this thesis), to modify the public key field in the document standard and to change the signature scheme used by the issuer, without changing the actual presentation protocol, and has almost no impact on the prover and verifier performance, which is a very good result considering the security improvement we get.

Notice that, due to the design direction we choose, our solution is not post-quantum secure in its entirety, since the device binding signature is still based on ECDSA, but at least the credentials themselves are protected against forgery by quantum adversaries, which is a very good improvement considering the actual threat model of the EUDI wallet and the fact that the device binding signature is only used to protect against replay attacks and not to protect the credential itself. A detailed security analysis of this claim is provided in the threat model section section 4.4.3.

4.2 Implementation

Given the design choices we made in the previous section, we decided to adopt the Ligerio framework for our solution as the main actor, for the following reasons:

- Open source code, which allows us to modify the code and adapt it to our needs;
- Ligerio is a zk proof system based on the MPC-in-the-head paradigm, which is by itself post quantum secure;
- Ligerio has been optimized for heavy circuits like ECDSA signature verification circuit or sha256 preimage circuit, which are the main bottlenecks of our solution;

In the following paragraphs we are going to describe the attempts we made that brought us to the final choice of using Ligerio as main actor together with zk-dilithium library and MAC code witness passing technique.

ML-DSA device binding This direction was explored in the early stage of the thesis. The idea was to replace the device binding signature scheme (ECDSA) with a post-quantum NIST standardized signature scheme, ML-DSA [28]. There were 2 main problems here, the first one is the compliance with LoA High treated in section 2.4. LoA high requires that the device binding signature is generated inside a tamper resistant secure element, which is not possible with ML-DSA which is not believed to be implementable in current devices. On this side of the problem, we actually found a paper [24] by Lee et al. that provides a valid implementation of ML-DSA signature inside Android secure element. The paper begins with a complete analysis of the android secure element and define different steps of the author’s approach to the implementation. The performance was not satisfactory at all for our purposes, and are reported in the table below 4.

The second problem is that ML-DSA is not a zk-friendly signature scheme, and it is difficult and not efficient at all to implement the signature verification algorithm inside Ligerio, which is the main actor of our solution. This problem made us move from this direction to the next one.

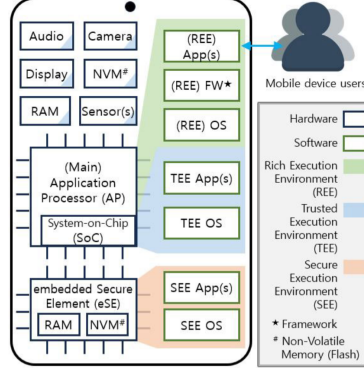


Figure 8: Secure element architecture.

Table 4: Total processing time comparison of drop-in replacement versus the proposed post-quantum cryptography migration scheme (values in milliseconds).

Category	Scheme	Total
ML-DSA-44	Drop-in replacement	1,351
	Proposed scheme	933
ML-DSA-65	Drop-in replacement	1,826
	Proposed scheme	1,229
ML-DSA-87	Drop-in replacement	2,119
	Proposed scheme	1,445

Post-quantum Ligerio signatures The next direction involved a full research on the actual state of the art of post-quantum signature schemes implementation. Still keeping in mind our design choice to be compliant with the actual standards, we focused on the possibility to implement other algorithm then ECDSA for the device binding signature, and other algorithm then ECDSA for the issuer signature. For the device binding signature, as Lehmann et al. pointed out in their paper Vision [25], current Secure Elements merely supports ECDSA signatures over P-256 curves. Moreover we needed the signature algorithm to be easily provable with zero knowledge technique in order to enable user to selectively disclose the identity attributes of the credentials.

With respect to Ligerio, no zero knowledge friendly post-quantum signature scheme has been implemented up to now. There were other proving systems like zkVM, for example RiscZero [32] that we tested in order to prove basic claims, but their run on android were extremely inefficient and not suitable for our purposes. We had to find another approach to solve the problem.

zkDilithium library This is where we had a fundamental intuition, which is actually one of the main contributions of this thesis to the current state of the art.

Claim 1. *Post-quantum security against a passive quantum adversary does not require the device binding signature algorithm to be post-quantum secure.*

We can show this by modifying the actual Ligerio circuit approach to the credential presentation problem. Since Ligerio is a zero knowledge proof system, in presentation moment the adversary is going to learn only that we know a valid device signature which involves the device public key contained in the credential. Revealing device public key to the relying party would lead to a privacy violation, since the relying party could link different presentations of the same credential. The device public key is also involved in another operation: credential issuance. In this case, a semi-honest issuer could store user credential and user's device public key, and later on could break the device key offline and clone his credentials on every smartphone, invalidating the device binding property. To solve this problem, we came out with the following solution: the user will not send the issuer his device public key, but will instead send the sha256 of the device public key, and will prove in zero knowledge that he knows a valid preimage of the sha256. This way, the issuer will not be able to break the device key offline, since he does not know the device public key, and the relying party will not be able to link different presentations of the same credential, since he does not know the device public key.

This direction allowed us to pass to the next step, which is to find a post-quantum signature scheme that is zk-friendly and can be used for the issuer signature. The most promising candidate we found is zk-dilithium [31], an open source library that implements a zero knowledge friendly version of dilithium, a NIST standardized post-quantum signature scheme. The paper's primary contribution is modifying the standard Dilithium (ML-DSA) algorithm to be efficient within a Zero-Knowledge proof system (specifically zkSTARKs). The main optimizations they introduce are:

- **Hash Function Replacement:** Standard Dilithium uses SHA-3/SHAKE, which is very expensive to prove in ZK. The paper replaces it with Poseidon, an algebraic hash function designed for STARK efficiency;
- **Public Key Distribution:** They do not use public key compression. By distributing the full vector t in the public key, they eliminate the need for the complex "hint" h in the signature verification process;
- **"Sample In Ball" Optimization:** The deterministic algorithm used to generate challenges is modified to avoid rejection sampling within the ZK circuit, significantly lowering the number of constraints needed.

Given this optimizations, the results that the authors report in their paper are the provided in the table 5 below.

Table 5: Run times and peak memory usage.

Scheme	Prover (ms)	Verifier (ms)	Prover RAM (MB)
Better size	4822	19.8	235
Balanced	660	22.0	40
Better time	304	31.4	19

The performance are completely compatible for our application, and the fact that the library is open source allows us to modify it and adapt it to our needs, which is a very good thing considering the fact that we need to connect it with Ligerio and we need to modify the public key field in the credential document standard.

Now the situation is the following: we have Ligerio zero knowledge proof system which is optimized for proving sha256 preimage and ECDSA, and we will use to prove device binding, disclose attributes ecc, and we have the zk-dilithium STARK prover which we will use to prove issuer’s signature over device credentials. The next question is how to connect the two proof systems, since we need to prove that the Ligerio credentials that we disclose are the same proven to be signed by the issuer in the STARK proof, and this is what we are going to show in the next two paragraphs.

SHA256 vs Poseidon This paragraph is going to show the problem we have to face in order to connect Ligerio and zk-dilithium proof systems via a common commitment scheme. Immediately, we notice that this commitment should have 2 properties:

- **Binding:** The binding property dictates that once the commitment is sent, the committer cannot alter the underlying message [12]. We need the commitment to be associated to a single credential in order not to allow a malicious prover to build the two different proofs over two different credentials;
- **Hiding:** The hiding property ensures that when a commitment is sent, the receiver learns absolutely nothing about the underlying secret value [12]. We need our commitment to be hiding, and in particular to involve some randomness, because if the commitment is deterministic, then the prover’s commitment across different presentations of the same credential would be the same, and this would allow the verifier to link different presentations of the same credential.

The first and simpler solution that came to our mind was to use a common, public sha256 commitment of the message signed by the issuer, which corresponds to the hash of the credential. In this way, we just needed a public commitment, the same in the common public input for both Ligerio and STARK proof system, and we could prove in Ligerio that the commitment is associated to the sha256 of the credential, and in STARK that the commitment is associated to the issuer signature over the credential. Moreover, we had already the implementation of the sha256 preimage prover in Ligerio, and, as Frigo and Shelat shows in thier paper [19], the sha256 preimage proof performance, that are reported in the table 6 below, are completely compatible with our application. The problem there became the following: how to implement sha256 inside

Table 6: Timing benchmark of an n-block pre-image to the SHA-256 function performed on x86_64 Emerald Rapids.

Blocks (n)	Verify Transcript (ms)	Total ZK Prover (ms)
1 block	6.07	10.7
2 blocks	12.1	19.8
4 blocks	24.3	35.2
8 blocks	49.8	67.9

the STARK prover system ? The answere was not trivial at all. We searched for an efficient implementation of sha256 inside a STARK prover system, and we found the winterfell sha256 implementation on github [23] which had good performance, but we found out a field incompatibility problem between the winterfell sha256 implementation which was running on $GF(2^{128})$, and the zk-dilithium STARK implementation, which is a fork of the standard STARK winterfell implementation by Facebook citeFacebookWinterfell running on $q = 2^{23} - 2^{20} + 1$ field. The other way around was to implement the Poseidon hash function, which is implemented by default in the STARK proof system and runs in a few milliseconds, being poseidon a zk-friendly hash function. We searched online for a Ligerio implementation of the Poseidon23 hash function, which was the STARK version, but we were didn’t find any implementation. Since the field in which Poseidon23 function was implemented in the STARK proof system was different from the field in which Ligerio was implemented, we decided to try to look for a simpler way to connect the two proof systems.

Notice that, whenever there will be an online implementation of Poseidon23 for Ligerio, or sha256 for STARK in $q = 2^{23} - 2^{20} + 1$ field, we will be able to easily modify our solution to use the same hash function for both proof systems, which could lead to a possible speed up.

MAC code witness passing technique In this final paragraph we show how to connect the two proof systems without using a common hash function, but using an information theoretic MAC like Frigo and

Shelat paper in section 3.4 of their paper [19], which allows us to pass the sha256 of the credential from the Ligerio circuit to the zk-dilithium STARK circuit in a secure way. The idea is the following: the prover generates a random key k_p and sends it to the verifier, who generates a public key k_v and send it back to the prover. The orchestrator computes the MAC as $MACm = a * x + b$ over the sha256 of the credential x , where a and b are obtained as $(a, b) = k$ with respect to the key $k = k_p \oplus k_v$. The prover shows in Ligerio that he knows a valid preimage of the sha256 of the credential, and that the MAC is correct with respect to k_p and k_v , and in STARK that he knows a valid signature of the issuer over a credential whose sha256 has a MAC equal to the one received from the orchestrator with respect to k_p and k_v . T

his way, we can securely pass the sha256 of the credential from Ligerio to zk-dilithium without using a common hash function, and without revealing any information about the credential itself.

Further proofs of security and formal implementation details are provided in the next sections.

4.3 Proof formalization

In this section we are going to formalize how our design of the prover and the verifier works.

4.3.1 Glossary of Symbols

Credential and session

MSO The user’s credential (Mobile Security Object), a byte-string laid out as an array of fields.

tr *Session transcript*: a unique, fresh value jointly produced by prover and verifier during the handshake. It binds the proof to one specific session (anti-replay / freshness) and is the nonce the device signs for device binding. Used in k_v derivation and in constraint **(L7)**.

now Current Unix timestamp, checked against the credential’s validity window **(L6)**.

attr The attribute value being disclosed.

id Index of *attr* inside *MSO*.

Credential hash and commitments

e_1 SHA256(*MSO*[0 : 183])—the message the issuer actually signed. Shared (privately) by both proofs.

R_S STARK-side Poseidon23 commitment to (k_p, e_1) . Locks the prover’s key share so e_1 cannot be swapped.

R_L Ligerio-side SHA256 commitment to (k_p, e_1) . Same purpose, on the other proof system.

MAC (links the two proofs)

k_p Prover’s secret key share, fresh per presentation (unlinkability).

k_v Verifier’s key share, derived as $SHA256(R_L \parallel R_S \parallel tr)$. Depends on *both* commitments, so it cannot be fixed before committing to e_1 .

a, b The two 128-bit halves of $k_v \oplus k_p$ (a = high, b = low); together they form the affine-MAC key.

m The **MAC tag** of e_1 : $m = a \cdot e_1 + b \in \mathbb{F}$. It is an authentication tag—it neither hides nor encrypts e_1 . The *same* m appears in both proofs and is the value the verifier cross-checks.

Keys and signatures

PK_{II} Issuer public key (Dilithium, post-quantum).

SK_{II} Issuer secret key (Dilithium).

sign Issuer’s Dilithium signature on e_1 .

pk_{dx}, pk_{dy} Device public key (the two elliptic-curve coordinates over P-256).

(r_2, s_2) Device’s ECDSA-P256 signature over the session nonce, proving device binding.

Device-auth black boxes (Frigo–Shelat)

- H The hash applied to $tr \parallel hdr$ before the device ECDSA check (left unspecified; specified by Frigo–Shelat).
- hdr The DeviceAuth header prepended to tr (left unspecified).

Outputs

- π_S, π_L The STARK and Ligero zero-knowledge proofs.
- $Pres$ The presentation (X_S, X_L, π_S, π_L) handed to the verifier.

4.3.2 Notation

Notation	Meaning
$x \parallel y$	concatenation of bit-strings x and y
$[cond]$	1 if $cond$ holds, 0 otherwise
$z = z_{hi} \parallel z_{lo}$	for $z \in \mathbb{F}$: z_{hi}, z_{lo} are the most-/least-significant 128-bit halves

4.3.3 Assumed Components

$SHA256 : \text{byte-string} \rightarrow \{0, 1\}^{256}$	
$Poseidon23 : \text{byte-string} \rightarrow \{0, 1\}^{256}$	
$H : \text{byte-string} \rightarrow \{0, 1\}^{256}$	(Frigo–Shelat device check, unspecified)
$hdr : \text{byte-string}$	(DeviceAuth header, unspecified)
$\text{rand_bytes}(n) : \mathbb{N} \rightarrow \{0, 1\}^{8n}$	(<code>rand.randombytes(n)</code>)
$\text{now_ts}() : () \rightarrow \mathbb{N}$	(<code>time.getTimestamp(0)</code> , current Unix time)
$\text{attr_at}(MSO, id) : \text{byte-string} \times \mathbb{N} \rightarrow \text{byte-string}$	the id -th attribute element of MSO

Dilithium (post-quantum signature scheme). The issuer key pair is produced by `Dil.KeyGen`; the issuer signs the credential hash with `Dil.Sign`; the relation **(S3)** checks the signature with `Dil.Verify`.

$$\begin{aligned} \text{Dil.KeyGen}() &\rightarrow (SK_{II}, PK_{II}) \\ \text{Dil.Sign}(SK_{II}, msg) &\rightarrow sign \\ \text{Dil.Verify}(sign, msg, PK_{II}) &\rightarrow \{0, 1\} \end{aligned}$$

ECDSA over P-256 (device binding). The device key pair is produced by `ECKG`; the device signs the session nonce $H(tr \parallel hdr)$ with `ECSign`; the relation **(L7)** checks the signature with `ECVerify`.

$$\begin{aligned} \text{ECKG}() &\rightarrow (sk_d, (pk_{dx}, pk_{dy})) \\ \text{ECSign}(sk_d, msg) &\rightarrow (r_2, s_2) \\ \text{ECVerify}((r_2, s_2), msg, (pk_{dx}, pk_{dy})) &\rightarrow \{0, 1\} \end{aligned}$$

4.3.4 Proof Systems

$$\begin{aligned} \text{Prove}(x, w) \rightarrow \pi \quad &\textbf{require } (x, w) \in \mathcal{R}; \quad \textbf{ensure } \text{Verify}(x, \pi) = 1, \\ \text{Verify}(x, \pi) \rightarrow b \quad &\textbf{ensure } b = 1 \Rightarrow \exists w'. (x, w') \in \mathcal{R}. \end{aligned}$$

Module STARK The STARK module proves issuer-credential validity and links the two proof systems via the affine MAC. We name its instance X_S and witness W_S .

$$\mathcal{R}_{\text{STARK}} = \left\{ \begin{array}{l} x := (PK_{II}, m, k_v, R_S) \\ w := (sign, e_1, k_p, a, b) \end{array} \left| \begin{array}{ll} R_S = \text{Poseidon23}(k_p \oplus e_1) \wedge & (\text{S1}) \\ a = (k_v \oplus k_p)_{\text{hi}} \wedge & (\text{S2a}) \\ b = (k_v \oplus k_p)_{\text{lo}} \wedge & (\text{S2b}) \\ m = a \cdot e_1 + b \wedge & (\text{S2c}) \\ \text{Dil.Verify}(sign, e_1, PK_{II}) = 1 & (\text{S3}) \end{array} \right. \right\}$$

- **(S1)** R_S is the STARK commitment opening of (k_p, e_1) .
- **(S2a),(S2b)** The MAC key halves a, b are asserted to equal the high/low halves of $k_v \oplus k_p$; an incorrect a or b makes the proof fail.
- **(S2c)** m is the affine MAC of e_1 under that key.
- **(S3)** $sign$ is a valid Dilithium issuer signature on e_1 .

Module Ligerio The Ligerio module proves attribute disclosure, credential-to-device binding, validity window, and device possession. We name its instance X_L and witness W_L .

$$\mathcal{R}_{\text{Ligerio}} = \left\{ \begin{array}{l} x := (attr, id, tr, now, m, k_v, R_L) \\ w := (r_2, s_2, MSO, pk_{dx}, pk_{dy}, k_p, e_1, a, b) \end{array} \left| \begin{array}{ll} e_1 = \text{SHA256}(MSO[0 : 183]) \wedge & (\text{L1}) \\ R_L = \text{SHA256}(k_p \oplus e_1) \wedge & (\text{L2}) \\ a = (k_v \oplus k_p)_{\text{hi}} \wedge & (\text{L3a}) \\ b = (k_v \oplus k_p)_{\text{lo}} \wedge & (\text{L3b}) \\ m = a \cdot e_1 + b \wedge & (\text{L3c}) \\ attr = attr_at(MSO, id) \wedge & (\text{L4}) \\ MSO[128 : 160] = \text{SHA256}(pk_{dx} \parallel pk_{dy}) \wedge & (\text{L5}) \\ \text{int}(MSO[48 : 56]) < now < \text{int}(MSO[56 : 64]) \wedge & (\text{L6}) \\ \text{ECVerify}(r_2, s_2, H(tr \parallel hdr), (pk_{dx}, pk_{dy})) = 1 & (\text{L7}) \end{array} \right. \right\}$$

- **(L1)** e_1 is the SHA256 hash of the credential MSO .
- **(L2)** R_L is the Ligerio commitment opening of (k_p, e_1) .
- **(L3a),(L3b),(L3c)** Same structure as **(S2a)–(S2c)**: a, b are the high/low halves of $k_v \oplus k_p$, and $m = a \cdot e_1 + b$.
- **(L4)** $attr$ is the disclosed attribute at position id in MSO .
- **(L5)** The device public key (pk_{dx}, pk_{dy}) hashes to the value stored in MSO .
- **(L6)** The current time now lies within the credential's validity window.
- **(L7)** (r_2, s_2) is a valid device ECDSA-P256 signature over the session nonce (device possession / binding).

4.3.5 Algorithm 1 — Orchestrator

The Orchestrator as a prover.

$$\mathcal{R}_{\text{Orch}} = \left\{ \begin{array}{l} x := (PK_{II}, attr, id, tr, now, m, k_v, R_S, R_L) \\ w := (sign, r_2, s_2, MSO, pk_{dx}, pk_{dy}, e_1, k_p, a, b) \end{array} \left| \begin{array}{ll} k_v = \text{SHA256}(R_L \parallel R_S \parallel tr) \wedge & (\text{O1}) \\ (X_S, W_S) \in \mathcal{R}_{\text{STARK}} \wedge & (\text{O2}) \\ (X_L, W_L) \in \mathcal{R}_{\text{Ligerio}} & (\text{O3}) \end{array} \right. \right\}$$

where the sub-instances and sub-witnesses are the projections

$$\begin{aligned} X_S &= (PK_{II}, m, k_v, R_S), & W_S &= (sign, e_1, k_p, a, b), \\ X_L &= (attr, id, tr, now, m, k_v, R_L), & W_L &= (r_2, s_2, MSO, pk_{dx}, pk_{dy}, k_p, e_1, a, b). \end{aligned}$$

Remark 1. The only constraint not already enforced by $\mathcal{R}_{\text{STARK}}$ or $\mathcal{R}_{\text{Ligero}}$ is **(O1)**, the derivation of the verifier key k_v . Constraint **(O1)** binds both commitments R_S, R_L and the session transcript tr together, so k_v cannot be fixed before committing to e_1 . Every other property—the issuer signature **(S3)**, the device signature **(L7)**, the affine MAC **(S2c)/(L3c)**, the validity window **(L6)**, and so on—is inherited from the two sub-relations. Because $\mathcal{R}_{\text{Orch}}$ uses a single shared witness, the values e_1, k_p, a, b, m and k_v are automatically forced to agree across the STARK and Ligero instances.

Algorithm 1 Orchestrator (prover for $\mathcal{R}_{\text{Orch}}$)

Input: MSO , tr , device signature (r_2, s_2) , issuer signature $sign$, device public key (pk_{dx}, pk_{dy}) , attribute $attr$ with index id , issuer public key PK_{II}

Output: $Pres = (X_S, X_L, \pi_S, \pi_L)$

```

1:  $e_1 \leftarrow \text{SHA256}(MSO[0 : 183])$ 
2:  $k_p \leftarrow \text{rand\_bytes}(32)$ 
3:
4:  $R_S \leftarrow \text{Poseidon23}(k_p \oplus e_1)$ 
5:  $R_L \leftarrow \text{SHA256}(k_p \oplus e_1)$ 
6:  $k_v \leftarrow \text{SHA256}(R_L \parallel R_S \parallel tr)$  ▷ establishes (O1)
7:
8:  $a \leftarrow (k_v \oplus k_p)_{\text{hi}}$ 
9:  $b \leftarrow (k_v \oplus k_p)_{\text{lo}}$ 
10:  $m \leftarrow a \cdot e_1 + b$ 
11:
12:  $now \leftarrow \text{now\_ts}()$ 
13:
14:  $X_S \leftarrow (PK_{II}, m, k_v, R_S)$ 
15:  $W_S \leftarrow (sign, e_1, k_p, a, b)$ 
16:  $X_L \leftarrow (attr, id, tr, now, m, k_v, R_L)$ 
17:  $W_L \leftarrow (r_2, s_2, MSO, pk_{dx}, pk_{dy}, k_p, e_1, a, b)$ 
18:
19: ▷ Pre-condition (cf.  $\mathcal{R}_{\text{Orch}}$ ):  $(X_S, W_S) \in \mathcal{R}_{\text{STARK}}$  and  $(X_L, W_L) \in \mathcal{R}_{\text{Ligero}}$ 
20:  $(\pi_S, \pi_L) \leftarrow \text{parallel}(() \mapsto \text{STARK.Prove}(X_S, W_S), () \mapsto \text{Ligero.Prove}(X_L, W_L))$ 
21: return  $(X_S, X_L, \pi_S, \pi_L)$ 

```

4.3.6 Algorithm 2 — Verifier

What the verifier checks. On receiving a presentation $Pres = (X_S, X_L, \pi_S, \pi_L)$ and session transcript tr , the verifier accepts iff a witness for the Orchestrator relation $\mathcal{R}_{\text{Orch}}$ exists for the implied statement. Concretely it verifies the four properties below; the first that fails causes rejection.

- V1. Consistency.** Both proofs share the same MAC tag m , the same verifier key k_v , and the same session transcript tr ($m_S = m_L$, $k_{v,S} = k_{v,L}$, $tr_L = tr$). This forces the two instances to be the projections X_S, X_L of a single Orchestrator instance, preventing an adversary from mixing components of different sessions or credentials.
- V2. Binding of k_v (constraint **(O1)**).** The verifier key is the hash of *both* commitments together with the transcript, $k_v = \text{SHA256}(R_L \parallel R_S \parallel tr)$. This is exactly the additional constraint of $\mathcal{R}_{\text{Orch}}$ and is the one property that neither sub-proof checks internally.
- V3. STARK validity (constraint **(O2)**).** The proof π_S is a valid proof of knowledge for X_S : there exists W_S with $(X_S, W_S) \in \mathcal{R}_{\text{STARK}}$. This certifies the issuer Dilithium signature on e_1 and the correct MAC computation.
- V4. Ligero validity (constraint **(O3)**).** The proof π_L is a valid proof of knowledge for X_L : there exists W_L with $(X_L, W_L) \in \mathcal{R}_{\text{Ligero}}$. This certifies attribute disclosure, device binding, and credential freshness.

Conditions **V1**–**V4** together establish that there exists a witness W_{Orch} with $((X_S \cup X_L), W_{\text{Orch}}) \in \mathcal{R}_{\text{Orch}}$: **V1** identifies the shared public values, **V2** discharges the additional constraint **(O1)**, and **V3**–**V4** discharge **(O2)**–**(O3)**.

Algorithm 2 Verifier

Input: $Pres$, session transcript tr **Output:** accept or reject

```
1: parse  $Pres = (X_S, X_L, \pi_S, \pi_L)$ 
2: parse  $X_S = (PK_{II}, m_S, k_{v,S}, R_S)$ 
3: parse  $X_L = (attr, id, tr_L, now, m_L, k_{v,L}, R_L)$ 
4:
5:  $\triangleright$   $V1$ : consistency — shared MAC, verifier key, and session
6: if  $m_S \neq m_L$  then return reject
7: end if
8: if  $k_{v,S} \neq k_{v,L}$  then return reject
9: end if
10: if  $tr_L \neq tr$  then return reject
11: end if
12:
13:  $\triangleright$   $V2$ : binding — constraint (O1) of  $\mathcal{R}_{Orch}$ 
14: if  $k_{v,S} \neq \text{SHA256}(R_L \parallel R_S \parallel tr)$  then return reject
15: end if
16:
17:  $\triangleright$   $V3$ : STARK proof validity — constraint (O2)
18: if  $\text{STARK.Verify}(X_S, \pi_S) \neq 1$  then return reject
19: end if
20:
21:  $\triangleright$   $V4$ : Ligero proof validity — constraint (O3)
22: if  $\text{Ligero.Verify}(X_L, \pi_L) \neq 1$  then return reject
23: end if
24: return accept
```

4.4 Security Analysis

In this section we are going to perform analysis with respect to the security of our solution, we are going to analyze the compliance of our solution with the eIDAS 2.0 regulation and with the LoA High requirements, and we are going to analyze the threat model of our solution, showing how and why it satisfies our design choice explained in section 4.1.

4.4.1 MAC witness sharing

STARK zk proof over the issuer signature needs dilithium verification algorithm. The algorithm takes in input the signature sig , the message being signed e_1 and the public key of the issuer PK_{II} . The message signed by the issuer e_1 is the sha256 of the credential document. Knowledge of sha256 preimage is hard to prove in STARK winterfell circuit, so we propose the following solution: the Ligero circuit, which has an optimized sha prover, will prove $e_1 = \text{sha256}(cred)$, then e_1 is encoded in a binding and hiding commitment realized with information theoretic MAC.

Mathematical proof of MAC security:

$$m = \text{MAC}_k(e_1) = \text{MAC}_{k_p, k_v}(e_1) = \text{MAC}_{a, b}(e_1)$$

We obtain the key as:

$$(a, b) = k_p \oplus k_v$$

where k_p is a secret random 32 bytes key chosen by the prover and k_v is a public verifiable key chosen by the verifier. What security concerns do we have?

1. The prover can try to forge a valid MAC for a different e'_1 , but e'_1 would still need to be a valid sha256 of signed credential since the signature is verified by the circuit;
2. The prover could generate k_p after seeing k_v , leading to arbitrary proves;
3. Different presentations could be linkable to the same MAC.

2 is avoidable if we require the prover to generate k_p before seeing k_v , committing to and building k_v with fiat shamir transform on k_p commitment, and 3 is avoidable if we require the prover to generate a different k_p for each presentation.

For 1, we have the following security proof in which we show that is not possible to forge a valid MAC for a different e'_1 without breaking the security of the MAC scheme:

$$m = MAC_k(e_1) = a * e_1 + b = a * e'_1 + b \Rightarrow a * (e_1 - e'_1) = 0$$

which is true either if $a = 0$ or if $e_1 = e'_1$. It has been proven that the probability that $a = 0$ is negligible. if $e_1 = e'_1$ then the prover is proving the same credential, so no forgery is happening. We also now that is not possible to have $m_1 = a' * e_1 + b' = m_2 = a'' * e'_1 + b''$ since the prover is not able to choose a and b due to the verifier participation in the key generation with k_v .

4.4.2 Compliance with eIDAS 2.0 and LoA High

In this section we are going to analyze our solution, taking into account the requirements of the European Commission for the EUDI wallet application, considering both the eIDAS 2.0 regulation and the LoA High (Level of Assurance High).

Dynamic Authentication and Replay Protection

CIR 2015/1502 requires that for LoA High, authentication must be dynamic, creating a proof that changes with each session. The plan fulfills this by including a session transcript (tr) in the Device Binding proof. The device's Secure Element signs this transcript, and the ZK circuit verifies this signature (p256.verify), ensuring the proof cannot be replayed by an attacker who has intercepted a previous proof.

Hardware Binding

The plan enforces Device Binding by extracting the public key (pk_d) from the signed MSO and requiring a proof of possession of the corresponding private key. This ensures the credential is bound to a tamper-resistant WSCD (Wallet Secure Cryptographic Device).

Algorithmic Soundness

The extraction error of the combined STARK/Ligero system is negligible ($O(\eta + d \cdot \log(w) / |F|)$), providing a robust mathematical barrier against forgery.

Compliance with EUDI ARF Requirements

The implementation satisfies several High-Level Requirements (HLRs) from the EUDI Architecture and Reference Framework:

- **User Sole Control:** The ZKP generation happens entirely on the user's device (orchestrated by the Wallet Unit), ensuring the user has sole control over attribute disclosure;
- **Validity Verification:** The circuit enforces validity checks ($t_{start} < t_{now} < t_{end}$) without revealing the actual issuance or expiration dates, as required for privacy-preserving interactions;

4.4.3 Threat Model

In this section we clarify the threat model we are considering for our solution, and explaining clearly where this thesis is going to be placed inside the context of the EUDI wallet application.

Current implementations completely lacks of post-quantum security, since current infrastructures relies on RSA and ECDSA signatures, and since hardware constraints of the LoA High requirements of the eIDAS 2.0 regulation forces the use of P256 curve. Since now, for this reasons, no one has already made any steps in the direction of getting a post-quantum solution for the EUDI wallet application.

With the solution proposed in this thesis, we are not achieving a full post-quantum secure wallet: instead, what we are achieving is post-quantum security for the current state of the art and technology advance in the quantum computing research. In this moment, no quantum computer is able to break RSA nor ECDSA signatures, but what an adversary could do is to record authentication sessions as a relying party or just a passive network monitoring adversary, and later on get the capability to break the signatures and forge device binding and issuer signature, or worst. A malicious issuer could collect users device public key and credentials, and later on break that set of keys and, being able to forge also device binding signature, impersonate other users.

With our solution we are protecting millions of users from this kind of risks. In fact, this solution is completely secure against semihonest adversaries, as shown before. Why our solution is not actual secure against malicious adversaries?

Forgery attack A malicious prover, which owns a valid credential and owns a quantum computer, could reverse engineer the application and extract the public key of the device. This is possible because, when the proof is generated, the device public key is needed in order to generate the zero-knowledge proof for the validity of the device binding signature. This operation, given the actual hardware restriction, cannot be made inside the Secure Element, therefore the device public key is exposed to the wallet unit which runs the prover algorithm, and with proper technique could be extracted and reversed with the Shor’s algorithm [34] in order to obtain the device private key.

Mitigation This attack allows a malicious prover to extract the device public key from his own device. In order for the attacker to be able to use the stolen device keys, he will still need the credentials. In the particular (and easier) case in which the attacker is the owner of the device and the credentials, he will be able to pass his credentials across different deiveces and therefore he could allow other users to use his credentials. This violates the LoA High required by the eIDAS 2.0 regulation, but does not allow the attacker to impersonate other users, since he will not be able to forge device binding signature for other users. Instead the attacker is only able to share his identity, which in first place harm his own privacy. There is also a sever economic disincentive for the attacker to forge his own device keys, sincea this implies a quantum computer attack, expensive and time consuming, in order to forge a single keypair, which by the way is his own keypair.

In conclusion, the solution proposed in this thesis is secure against semihonest quantum adversarial prover, but is a perfect bridge between a completly quantum-agnostic implementation of the wallet and a future post-quantum sound implementation of the wallet, which will be able to protect users against malicious quantum adversaries. Post-quantum sound solutions are likely to be out soon, since the enitre online comunity, due to incurrent quantum threat, is moving towards post-quantum secure softwares. In particular, Google declared that the next version of their operating system, Android 17, will implement post-quantum secure NIST standardized algorithm ML-DSA [20].

5 Results

6 Future Work

The prototype presented in this thesis demonstrates, for the first time, that privacy-preserving, post-quantum, device-bound anonymous credentials are feasible on commodity hardware. It remains, however, a proof of concept: it shows that the approach *can* work, not that it is yet a complete, formally verified, and standards-ready system. The remainder of this section details the concrete directions along which this work can and should be extended.

6.1 Formal Treatment and System Completion

The most immediate gap between the current prototype and a deployable system is the lack of a formal security treatment. Future work should provide a formal security model for post-quantum, device-bound anonymous credentials operating on legacy hardware and legacy infrastructures, together with reductions to well-established hard problems for each underlying construction. Closing this gap would not only place the scheme on rigorous theoretical footing but would also make it, to the best of current knowledge, the first concrete instantiation of an EUDI Wallet credential that simultaneously fulfils all of the European Commission’s stated requirements for the wallet, rather than approximating a subset of them as current deployable approaches do.

6.2 Optimization of the Mixed Zero-Knowledge Construction

The present construction is built by mixing two different zero-knowledge proof technologies: a hash-based zero-knowledge bridge on top of a classical, secure-element-bound ECDSA signature, composed with an outer post-quantum signature layer. This composition was designed primarily to demonstrate feasibility, and a closer examination of *how* the two proof systems are mixed and composed is likely to yield a substantial performance improvement. Future work should:

- profile the zero-knowledge layer end to end to locate the actual bottlenecks, including circuit size, the choice of commitment scheme, the underlying field, and the hash function used;
- iterate on the proof system itself and on the way its components are composed, rather than treating the two ZK technologies as a fixed black-box pipeline;
- re-benchmark proving time, verification time, and proof size after each optimization round, comparing against the original prototype and against external baselines.

Concrete target metrics for this optimization effort include sub-second presentation proving on mid-range phones, proof and communication sizes competitive with the Frigo–Shelat construction underlying Google’s Longfellow system, and explicit coverage of low-end devices that lack a high-assurance secure element, which the current prototype does not specifically address.

6.3 Extended Credential Functionality

Beyond the core presentation proof over the standard MDOC document format, several features are necessary for a credential system to be realistically deployable at the scale of a national or European digital identity wallet, and none of them are trivial extensions of the base scheme:

- **Multiple-attribute presentation.** Support presenting several attributes in a single proof, including attributes drawn from different credentials, assembled and proved jointly rather than one credential at a time.
- **Revocation lists.** Implement a revocation mechanism that lets a verifier efficiently check the current validity of a credential after the issuer has invalidated it. Revocation is essential for any real-world deployment, but is notoriously difficult to implement without breaking the unlinkability properties that anonymous credentials are meant to provide.
- **Pseudonyms.** Implement accountable pseudonyms that let a user log in to, and persist a stable identifier with, a given relying party for an arbitrary purpose, while remaining anonymous and unlinkable everywhere else. Pseudonym support is mandated by the EUDI Architecture Reference Framework, yet, at the time of writing, no advanced anonymous-credential system has produced a working implementation of it.
- **Presentation deniability.** Investigate mechanisms that always allow a user to deny having produced a specific presentation, optionally at the verifier’s discretion. Such deniability can be built on different paradigms, for instance on the property that a given proof could plausibly have been produced by someone else entirely, rather than necessarily by the holder of the credential.
- **Blind signature of credentials.** Explore blind-signature constructions that let an issuer sign credential data supplied by the user without learning what is actually being signed, while still being able to verify that the blinded data is well-formed and correct.
- **Social recovery.** Design a social-recovery mechanism, based on threshold cryptography and a set of trusted users, so that the loss of a device does not entail the unrecoverable loss of all the credentials and private information it held. This is particularly important in a system that, by design, avoids a central authority responsible for backing up user data.

6.4 Alternative and Exploratory Research Directions

A number of directions depart from the specific path taken in the current prototype but are worth investigating in their own right:

- **Multi-party computation (MPC).** Investigate whether efficient MPC protocols over the anonymous-credential system could circumvent, rather than work around, the limitations of secure elements that cannot natively execute the algorithms the scheme requires.
- **Native post-quantum support in future Android releases.** Android 17, expected in 2026/27, has been announced to implement the ML-DSA post-quantum signature standard. Once secure elements natively support ML-DSA, it becomes possible to combine a post-quantum device-binding signature with a post-quantum issuer signature efficiently and natively, removing the need for the hybrid hash-based bridge used in the current prototype.

- **Hash-based post-quantum signatures.** Legacy secure elements are already capable of computing the SHA-256 hash function, which is believed to be post-quantum resistant. This opens the possibility of implementing a standardized hash-based post-quantum signature scheme, such as SPHINCS+, directly inside the secure element, as a potentially more efficient alternative to ML-DSA in terms of proving time and key size.

6.5 Evolving the Threat Model

The design should continue to be evaluated against two distinct time horizons, and future work should make this evolution explicit rather than implicit:

- **Medium term.** No cryptographically relevant quantum computer exists yet, but the deployed infrastructure – secure elements limited to ECDSA over P-256, classical issuers and verifiers, and hundreds of millions of presentations already being made – does. The pressing danger in this horizon is store-now-decrypt-later and its privacy-specific variant, store-now-link-later: an issuer, a set of colluding verifiers, or a passive network observer can record today’s presentations and, once a quantum computer becomes available, retroactively break the classical assumptions that kept those presentations unlinkable, de-anonymizing users across services and over time. Because identity data remains sensitive for decades and a privacy breach cannot be undone, the relevant clock starts at the moment a presentation is made, not at the moment a quantum computer first appears, which is precisely why this threat must be addressed now, on hardware that cannot be changed.
- **Long term.** The infrastructure itself is eventually upgraded: secure elements gain native post-quantum signatures, and issuer and verifier stacks complete their NIST- and EU-mandated migrations. Future work should ensure the scheme is designed to grow into this setting without requiring a redesign, rather than being optimized exclusively for the medium-term, legacy-hardware case.

A related, but comparatively lower-priority, risk is credential forgeability. Because device binding requires the device’s public key to be embedded in the credential document, a sufficiently capable quantum adversary could in principle recover the corresponding ECDSA secret key and forge the binding. By itself, this only grants the ability to use an already-stolen credential, or to clone one’s own credential, and only for one device at a time; mounting this attack at any meaningful scale would require physically extracting keys from a very large number of individual phones, a cost that is not justified by any realistic gain. The timeline further weakens this risk: with ML-DSA support expected in Android secure elements from 2026, and roughly four years estimated before a cryptographically relevant quantum computer (Q-day) materializes, device binding should become post-quantum sound across most of the device base before this attack is ever practical. Future work should track this timeline and adjust the urgency of mitigations accordingly.

7 Conclusion

These are some benefits which derive from our solution proposed in this thesis:

- **Advancement over the state of the art.** Existing deployable approaches, such as Frigo–Shelat, do not achieve post-quantum privacy and remain dependent on the secure element; pairing-based BBS/BBS+ offers elegant selective disclosure but is neither post-quantum nor hardware-compatible. A system that simultaneously targets privacy preservation, post-quantum security, device binding, and deployability on legacy hardware, and validates all four properties on commodity phones, would be the first of its kind.
- **Impact on European Digital Identity.** Every EU Member State is required to offer an EUDI Wallet, and the regulation’s privacy goals are currently unmet. Directly addressing the two acknowledged weaknesses of the current design – over-disclosure and linkability – with real predicate proofs and unlinkable presentations, while making the wallet quantum-safe, would align it with both the EU post-quantum migration roadmap and the 2030 Digital Decade targets.
- **Societal benefits.** Citizens would gain genuine data minimization (for example, proving "older than 18" without revealing a birth date, or proving residence without revealing an address), protection from cross-service profiling, resilience against device loss through social recovery, and access regardless of phone tier, advancing both privacy-by-design and digital inclusion for hundreds of millions of users.

- **Economic and standardization impact.** Results would feed directly into EUDI ARF discussions, contributing to European sovereignty over a privacy-preserving, quantum-safe identity stack rather than importing one. An open-source library would lower the barrier for national wallet providers and for the private-sector relying parties the regulation aims to onboard, and would be reusable in adjacent settings such as Privacy Pass, remote attestation, and online age verification.
- **Deployment-friendliness.** Critically, this direction does not require citizens to purchase modern smartphones, nor does it require relying parties to adopt modern hardware. The only requirement on the ecosystem side is for the issuer to adopt a post-quantum, Dilithium-class standardized signature and to reduce the size of the device public-key field carried in the credential document, which, as a side effect, also yields a net space optimization in the document itself.

x

References

- [1] Heather Adkins and Sophie Schmieg. Quantum frontiers may be closer than they appear. Google Blog, 2026. Accessed: May 24, 2026.
- [2] Iván Abellán Álvarez, Pol Hölzmer, and Johannes Sedlmeir. Privacy evaluation of the European Digital Identity Wallet’s Architecture and Reference Framework. *Computers & Security*, 160, 2026.
- [3] Zahra Ebadi Ansaroudi, Roberto Carbone, Giada Sciarretta, and Silvio Ranise. Control is Nothing Without Trust: A First Look into Digital Identity Wallet Trends. In Vijayalakshmi Atluri and Anna Ferrara, editors, *Data and Applications Security and Privacy XXXVII*, volume 13942 of *Lecture Notes in Computer Science*, pages 113–132. Springer, Cham, 2023.
- [4] Zahra Ebadi Ansaroudi, Giada Sciarretta, Andrea De Maria, and Silvio Ranise. Navigating secure storage requirements for EUDI wallets: a review paper. *EURASIP Journal on Information Security*, 2025(1):2, 2025.
- [5] Ryan Babbush, Adam Zalcman, Craig Gidney, Michael Broughton, Tanuj Khattar, Hartmut Neven, Thiago Bergamaschi, Justin Drake, and Dan Boneh. Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations. Cryptology ePrint Archive, Paper 2026/625, 2026.
- [6] Christian Bormann and Anja Lehmann. SoK: Anonymous credentials for digital identity wallets. Cryptology ePrint Archive, Paper 2026/330, 2026. Security Standardisation Research (SSR) 2025.
- [7] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. Cryptology ePrint Archive, Paper 2001/019, 2001.
- [8] Jan Camenisch and Anna Lysyanskaya. Signature schemes and anonymous credentials from bilinear maps. In Matt Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, pages 56–72, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
- [9] Jan Camenisch and Els Van Herreweghen. Design and implementation of the idemix anonymous credential system. In *Proceedings of the 9th ACM Conference on Computer and Communications Security, CCS ’02*, page 21–30, New York, NY, USA, 2002. Association for Computing Machinery.
- [10] David Chaum. Blind signatures for untraceable payments. In David Chaum, Ronald L. Rivest, and Alan T. Sherman, editors, *Advances in Cryptology*, pages 199–203, Boston, MA, 1983. Springer US.
- [11] David Cooper, Stefan Santesson, Stephen Farrell, Sharon Boeyen, Russell Housley, and William Polk. Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile. RFC 5280, Internet Engineering Task Force, 2008.
- [12] Ivan Damgård. Commitment schemes and zero-knowledge protocols. *Lecture Notes in Computer Science*, pages 63–86, 1999.
- [13] dyne.org. Longfellow-eu, 2025.

- [14] European Commission. Proposal for a Regulation of the European Parliament and of the Council amending Regulation (EU) No 910/2014 as regards establishing a framework for a European Digital Identity. COM(2021) 281 final, 2021. Accessed: May 22, 2026.
- [15] European Commission. European Digital Identity Wallet Architecture and Reference Framework — Version 1.4.0, 2024. Accessed: May 22, 2026.
- [16] European Commission. Recommendation on a Coordinated Implementation Roadmap for the transition to Post-Quantum Cryptography. Commission Recommendation (EU) 2024/1101, 2024. Accessed: May 24, 2026.
- [17] European Parliament and Council. Directive (eu) 2015/2366 on payment services in the internal market (psd2), 2015.
- [18] Karla Friedrichs, Franklin Harding, Anja Lehmann, and Anna Lysyanskaya. Device-bound anonymous credentials with(out) trusted hardware. Cryptology ePrint Archive, Paper 2025/1995, 2025. To appear at EUROCRYPT 2026.
- [19] Matteo Frigo and abhi shelat. Anonymous credentials from ECDSA. Cryptology ePrint Archive, Paper 2024/2010, 2024.
- [20] Google. Security for the Quantum Era: Implementing Post-Quantum Cryptography in Android. Google Security Blog, 2026. Accessed: June 22, 2026.
- [21] IETF. Sd-jwt-based verifiable credentials (sd-jwt vc), 2024.
- [22] ISO/IEC. Iso/iec 18013-5:2021 personal identification — iso-compliant driving licence — part 5: Mobile driving licence (mdl) application, 2021.
- [23] iStrannik. winterfell-sha256. GitHub repository, 2026. Accessed: June 22, 2026.
- [24] Sungmin Lee, Hyungchul Jung, Nohik Heo, and Ted "Taekyoung" Kwon. Case study: PQC migration for Android Strongbox Keymaster on Samsung Galaxy mobile devices. *IEEE Access*, 14:25770–25782, 2026.
- [25] Anja Lehmann, Andrey Sidorenko, and Alexandros Zacharakis. Vision: A modular framework for anonymous credential systems. Cryptology ePrint Archive, Paper 2025/1981, 2025.
- [26] Anna Lysyanskaya. *Signature schemes and applications to cryptographic protocol design*. PhD thesis, Massachusetts Institute of Technology, 2002.
- [27] René Mayrhofer, Anja Lehmann, and abhi shelat. A brief note on cryptographic pseudonyms for anonymous credentials. *arXiv preprint arXiv:2510.05419*, 2025.
- [28] National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard. Federal Information Processing Standards (FIPS) Publication 204, 2024. Accessed: June 21, 2026.
- [29] OpenID Foundation. Openid for verifiable presentations, 2023.
- [30] Christian Paquin and Greg Zaverucha. U-prove cryptographic specification v1.1 (revision 3), December 2013. Released under the Open Specification Promise (<http://www.microsoft.com/openspecifications/en/us/programs/osp/default.aspx>).
- [31] Guru-Vamsi Policharla, Bas Westerbaan, Armando Faz-Hernández, and Christopher A. Wood. Post-quantum privacy pass via post-quantum anonymous credentials. Cryptology ePrint Archive, Paper 2023/414, 2023. Accessed: June 21, 2026.
- [32] RISC Zero. RISC Zero: The zero-knowledge virtual machine, 2026. Accessed: June 22, 2026.
- [33] Amir Sharif, Zahra Ebadi Ansaroudi, Giada Sciarretta, Daniela Pöhn, Majid Mollaeefar, Wolfgang Hommel, and Silvio Ranise. Protecting digital identity wallet: A threat model in the age of eIDAS 2.0. In *Risks and Security of Internet and Systems (CRiSIS 2024)*, Lecture Notes in Computer Science. Springer, 2024.
- [34] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a Quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

- [35] Trail of Bits. Google Longfellow security review. Technical Report, 2025. Accessed: June 18, 2026.
- [36] W3C. Verifiable credentials data model 2.0, 2024.
- [37] W3C. Web authentication: An api for accessing public key credentials level 3, 2024.
- [38] Bas Westerbaan. Cloudflare targets 2029 for full post-quantum security. The Cloudflare Blog, 2026. Accessed: May 24, 2026.